

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



SOFTWAREPROZESSMODELLE

SCRUM – und andere

Themen in dieser Vorlesung

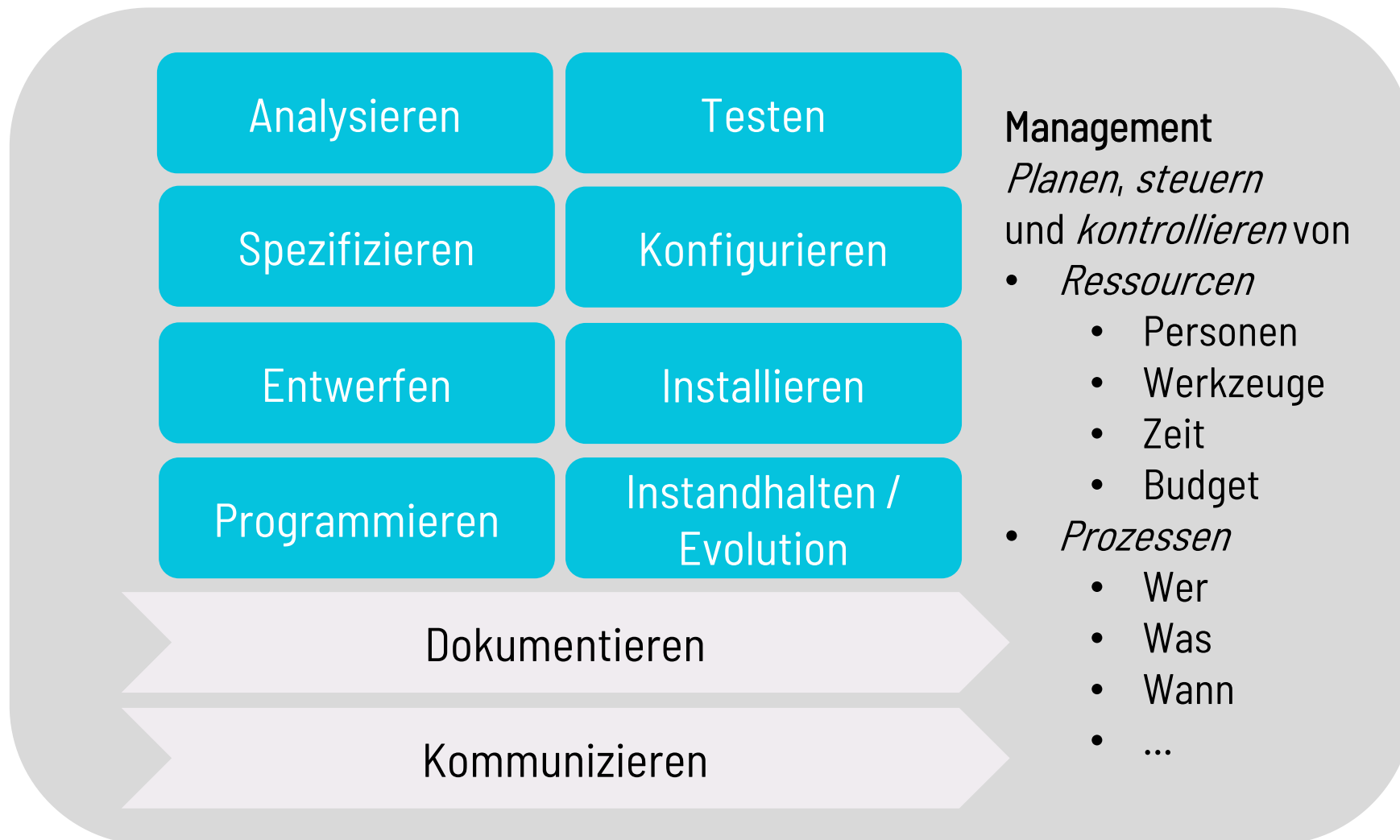


Lernziele

- Sie wissen, was einen Softwareprozessmodell ist, und kennen verschiedene Beispiele mit ihren Unterschieden
- Sie lernen Scrum als empirisches Vorgehensmodell in der Software-Entwicklung kennen
- Sie kennen die verschiedenen Rollen, Meetings und Artefakte in Scrum
- Sie können Scrum in einem kleinen Projekt selbständig anwenden



Wie genau durchführen? Von wem? Reihenfolge? Wie abgegrenzt?



Beschreibung von Prozessen

- Bei Beschreibung und Diskussion von Prozessen geht es hauptsächlich um **Aktivitäten** in diesen Prozessen, etwa Spezifikation des Datenmodells, Entwurf einer Benutzeroberfläche, ... und die **Reihenfolge** dieser Tätigkeiten.
- Prozessbeschreibungen können auch enthalten:
 - **Produkte** als Ergebnis einer Prozessaktivität;
 - **Rollen**, die für Verantwortlichkeiten von Personen im Prozess stehen;
 - **Vor- und Nachbedingungen**, also Aussagen, die vor oder nach einer Aktivität oder der Herstellung eines Produktes wahr sind.

Der Software-Prozess

Eine strukturierte Menge von Aktivitäten zur Entwicklung oder der Evolution eines Softwaresystems

- Wählt Aktivitäten des Software Engineering aus
- Gibt Anweisungen zur konkreten Durchführung
- Kombiniert Aktivitäten, ordnet in Reihenfolgen
- Benennt Rollen und Verantwortlichkeiten
- Dazu weitere Konventionen und Werte



Formale Darstellung im Software-Prozessmodell

Eine abstrakte Repräsentation eines Software-Prozesses unter Berücksichtigung unterschiedlicher Blickwinkel

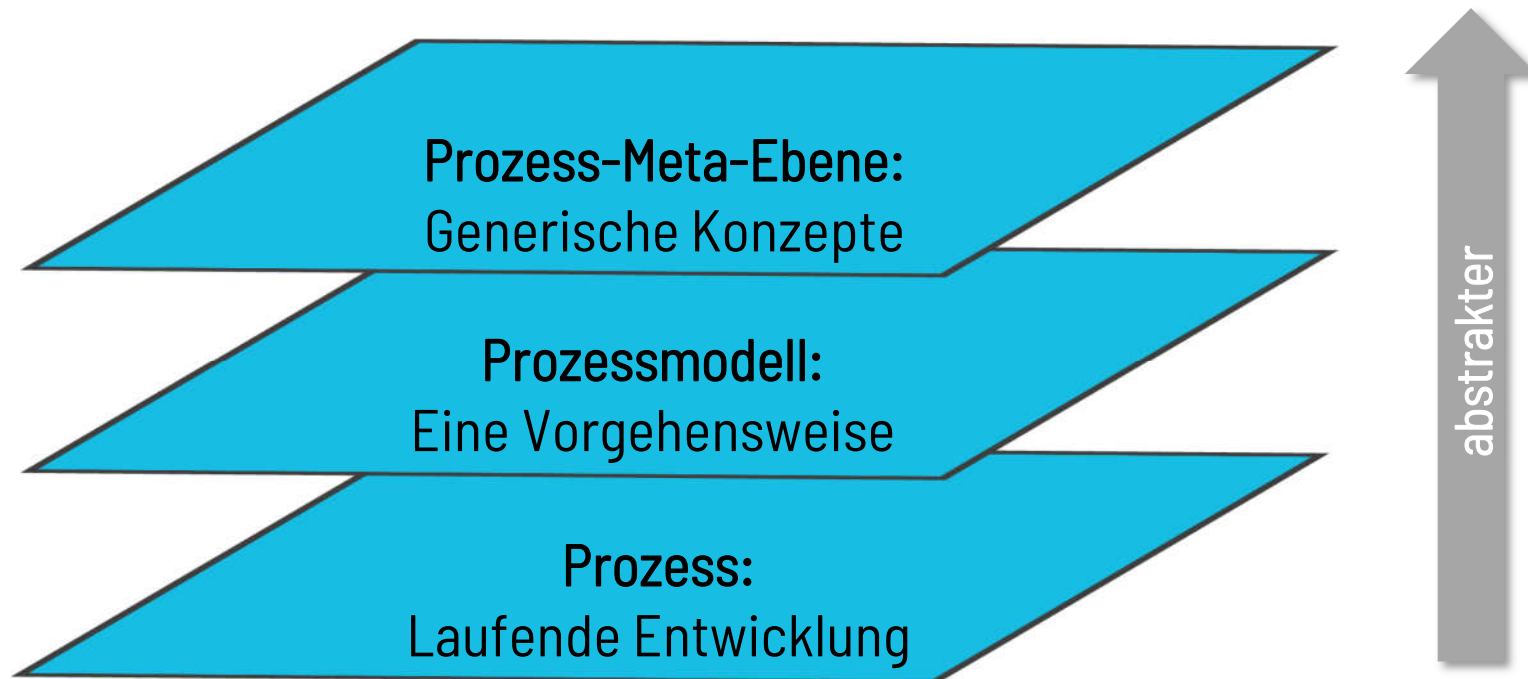
- Kontroll-, Datenfluss- oder Rollen-basierte Blickwinkel
- Allgemeine Prozessmodelle (auch "Vorgehensmodelle")
 - Wasserfall
 - Iterative Entwicklung
 - XP (eXtreme Programming)
 - RUP (Rational Unified Process)
 - Scrum
 - ...



Software-Prozessmodelle

- Software-Prozessmodelle werden auch „Vorgehensmodelle“ genannt
- Oft vereinfachende Sprechweise: „Prozess“, wo eigentlich „Prozessmodell“ gemeint ist.
 - „Der Wasserfall-Prozess...“
 - „XP ist ein Softwareprozess...“

Prozess <-> Prozessmodell



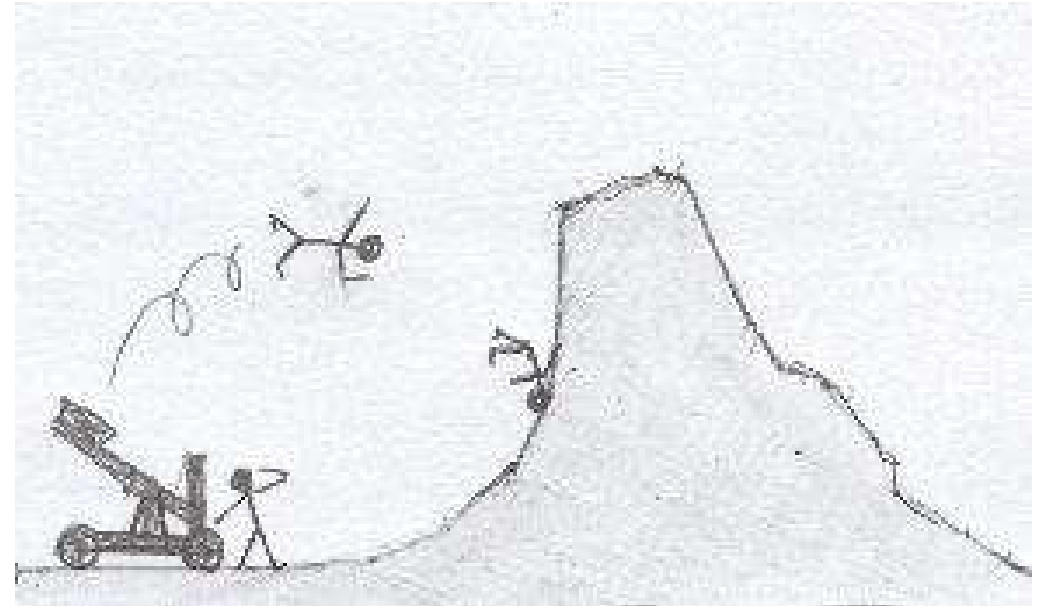
Plan-gesteuerte und agile Prozessmodelle

- Bei **plan-gesteuerten Prozessen** werden alle Aktivitäten vorweg geplant, Fortschritt wird gegen diesen Plan gemessen
- Bei **agilen Prozessen** wird in Abschnitten inkrementell (schrittweise hinzufügend) geplant. Deshalb ist es leichter, auf wechselnde Kundenanforderungen zu reagieren
- In der Praxis enthalten die meisten Prozesse sowohl plan-gesteuerte wie agile Elemente
- Es gibt keine falsche oder richtigen Softwareprozesse (nur falsch für den Einsatzzweck gewählte)

Code-and-Fix Modell in den 50-er und 60-er

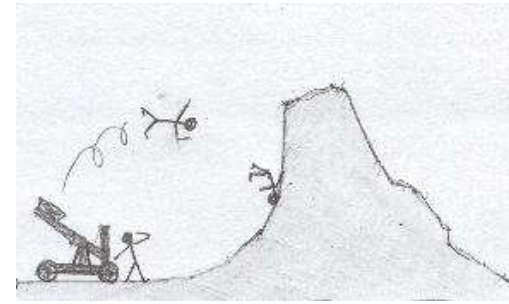
Direkte Codeerstellung ohne
nachhaltige Planung mit
anschließender
Fehlerbehebung

1. Schreibe ein Programm
(*Code*)
2. Finde und behebe die Fehler
im Programm (*Fix*)



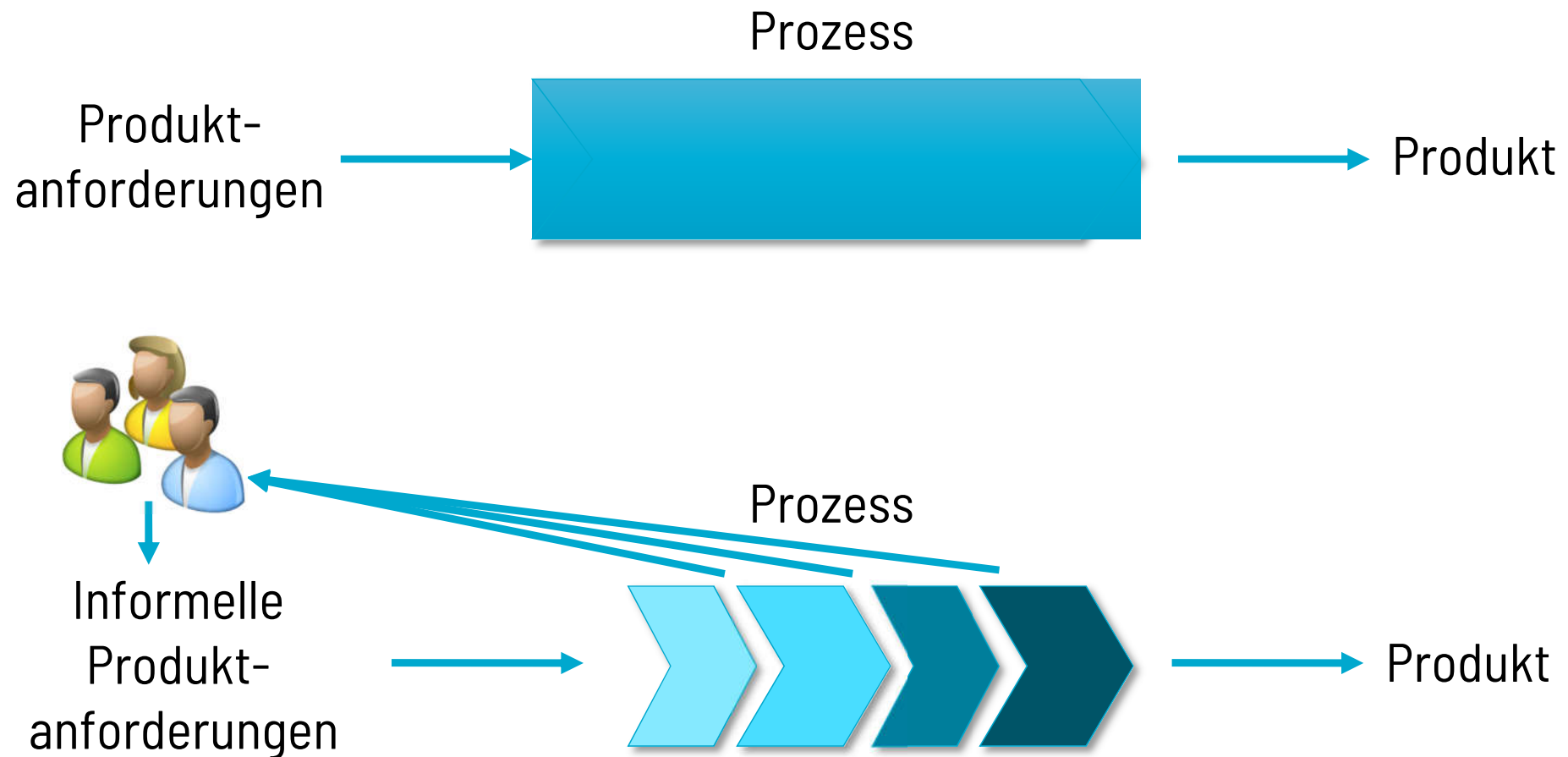
-- Christophe Thibaut
„Code And Fix - one year of slamming code,
one year of debugging.“

Diskussion: Code-and-Fix Modell

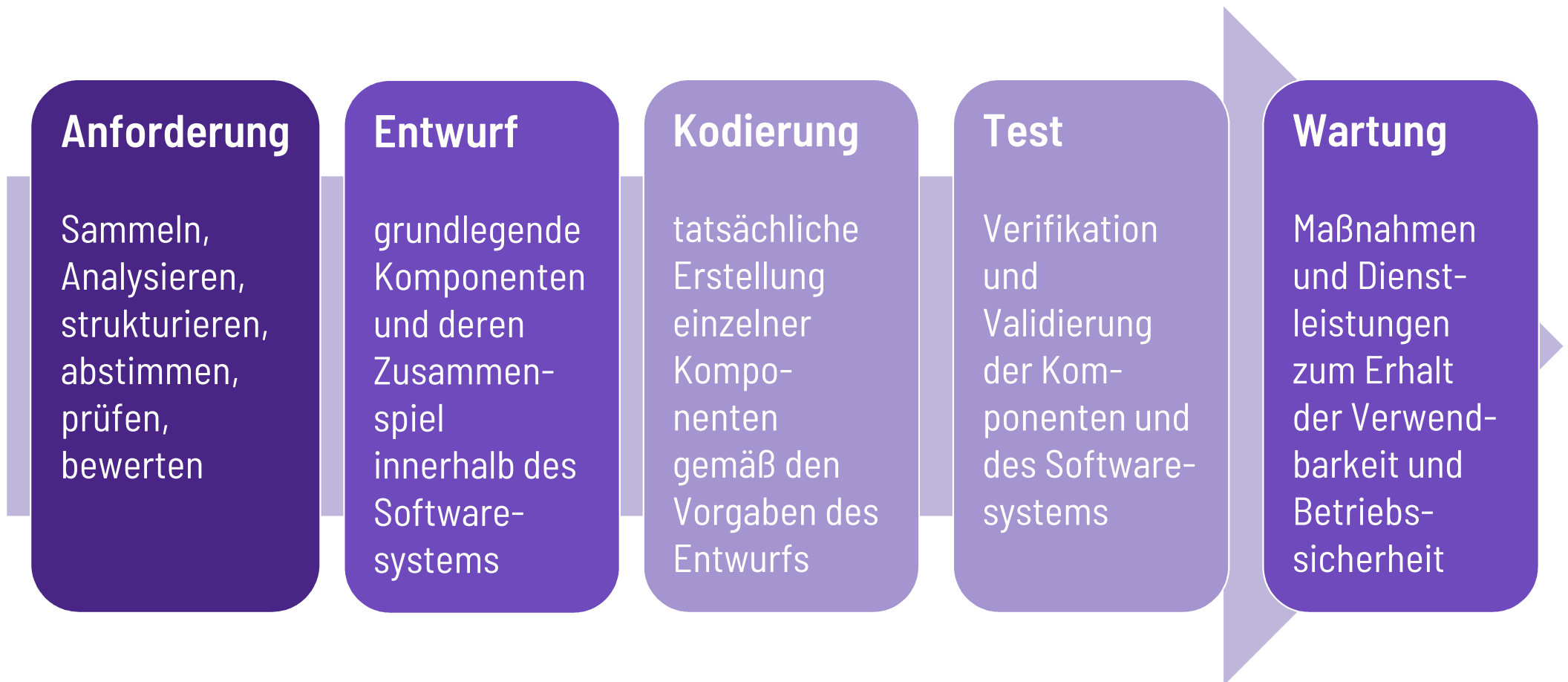


- Starke Abhängigkeit in allen Belangen vom Entwickler
- Geringe Akzeptanz beim Benutzer selbst bei gut entworfener Software
 - Einführung einer expliziten *Anforderungsphase* (*requirements-first process*)
- Fehlersuche sehr schwierig
 - Einführung einer *Testphase*
- Fehlerbeseitigungen führen zu grundlegenden (und sehr teuren) Unstrukturiertheiten im Code
 - Einführung einer separaten *Designphase*

Historische Herausbildung von Phasen

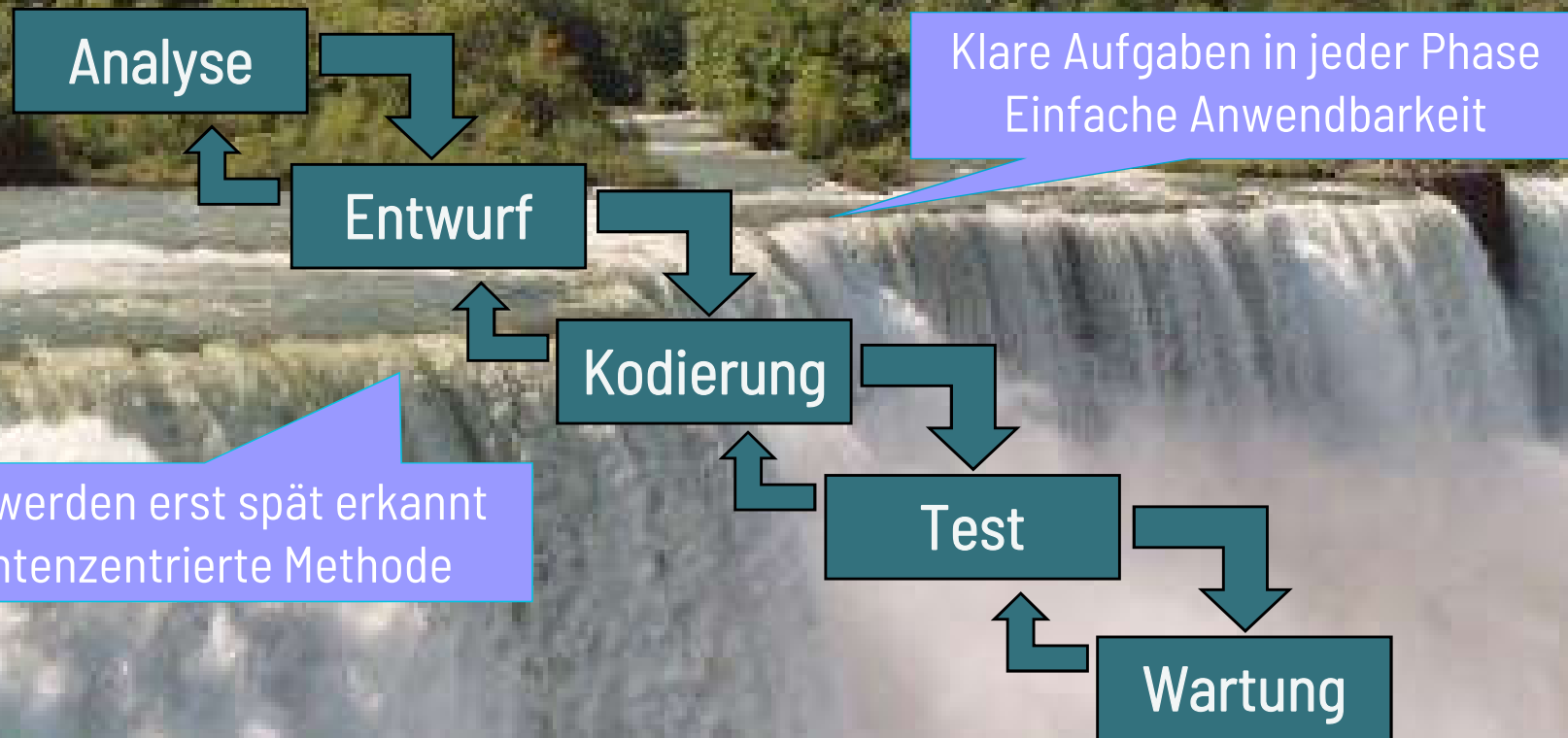


Herausbildung von Phasen



Wasserfall-Modell nach Royce

aus den 70-er



Wasserfall-Modell: Eigenschaften

- Jede Aktivität einer Phase muss in der richtigen Reihenfolge und in der kompletten „Breite“ vollständig durchgeführt werden
 - Ablauf ist strikt sequentiell
- Am Phasenende wird das Ergebnis der Aktivitäten schriftlich dokumentiert
 - Dokumenten-getriebenes Modell
- Vorgehen basiert auf Top-Down-Ansatz
- Generell einfaches, verständliches Vorgehen, welches moderaten Managementaufwand erfordert



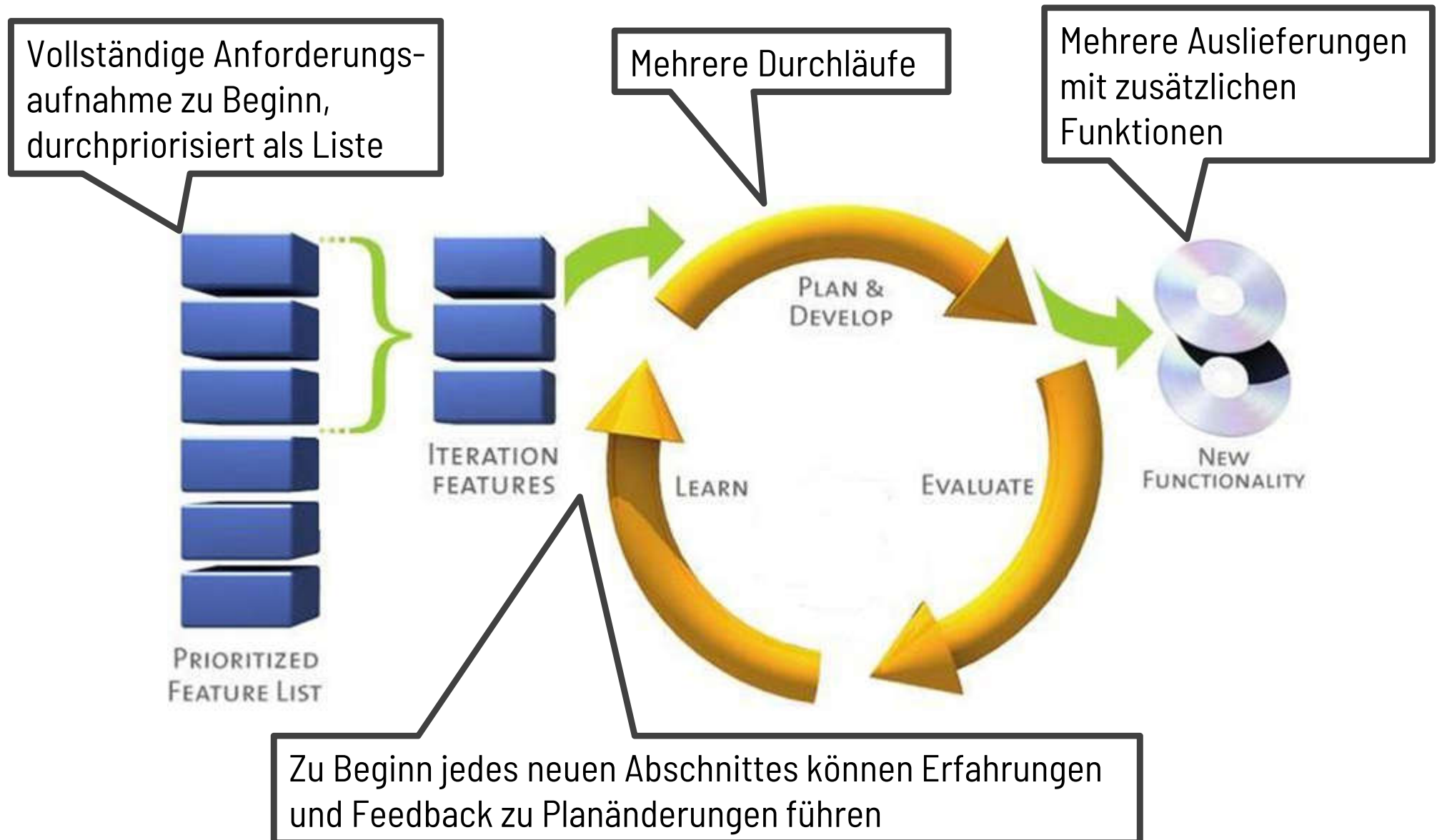
Gut geeignet für kleinere Projekte und Standardprojekte
Annahme: stabiles Umfeld und bekannte Technologien

Wasserfall-Modell: Kritische Punkte

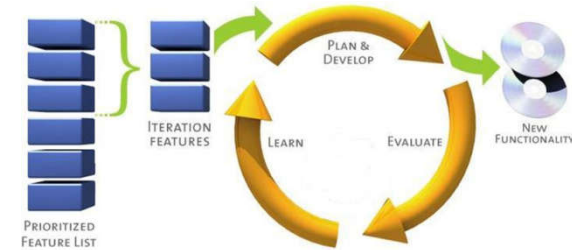
- In Realität sind Abläufe nicht immer strikt sequentiell durchzuführen
 - Abläufe sind häufig zu unflexibel
 - Abläufe nicht zu parallelisieren bzw. beschleunigen
- Gefahr einer Dokumentenflut
- Risikofaktoren werden unangemessen berücksichtigt
- Änderungsmanagement sehr aufwendig und teuer
- Lauffähige Version erst am Ende der Entwicklung
- Involvierung der Benutzer lediglich zu Beginn in der Anforderungs- bzw. Definitionsphase



Inkrementelle Softwareprozess-Modelle

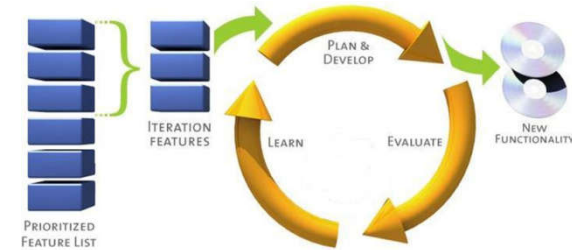


Inkrementelle Modelle: Eigenschaften



- Gleichmäßiger Fortschritt in kleinen Schritten
- Frühe Verfügbarkeit des Softwaresystems
- Frühe Erfolgserlebnisse
- Frühe Inkremente (bzw. „lauffähige Produktversionen“) erleichtern die Phase der Anforderungsdefinition deutlich
- Geringere Gefahr des „Scheiterns“, da Projektfortschritt transparent für alle Beteiligten (erleichterte Planung und Planverfolgung)
- Mögliche Fehler werden frühzeitig erkannt und Gegenmaßnahmen können zeitnah eingeleitet werden
- Realisierung wichtiger Funktionalitäten zuerst

Inkrementelle Modelle: Eigenschaften



- Gleichmäßiger Fortschritt in kleinen Schritten
- Frühe Verfügbarkeit des Softwaresystems
- Frühe Erfolgserlebnisse
- Frühe Inkremente (bzw. „lauffähige Produktversionen“) erleichtern die Phase der Anforderungsdefinition deutlich
- Geringere Gefahr des „Scheiterns“ da Projektfortschritt

trans
Planv

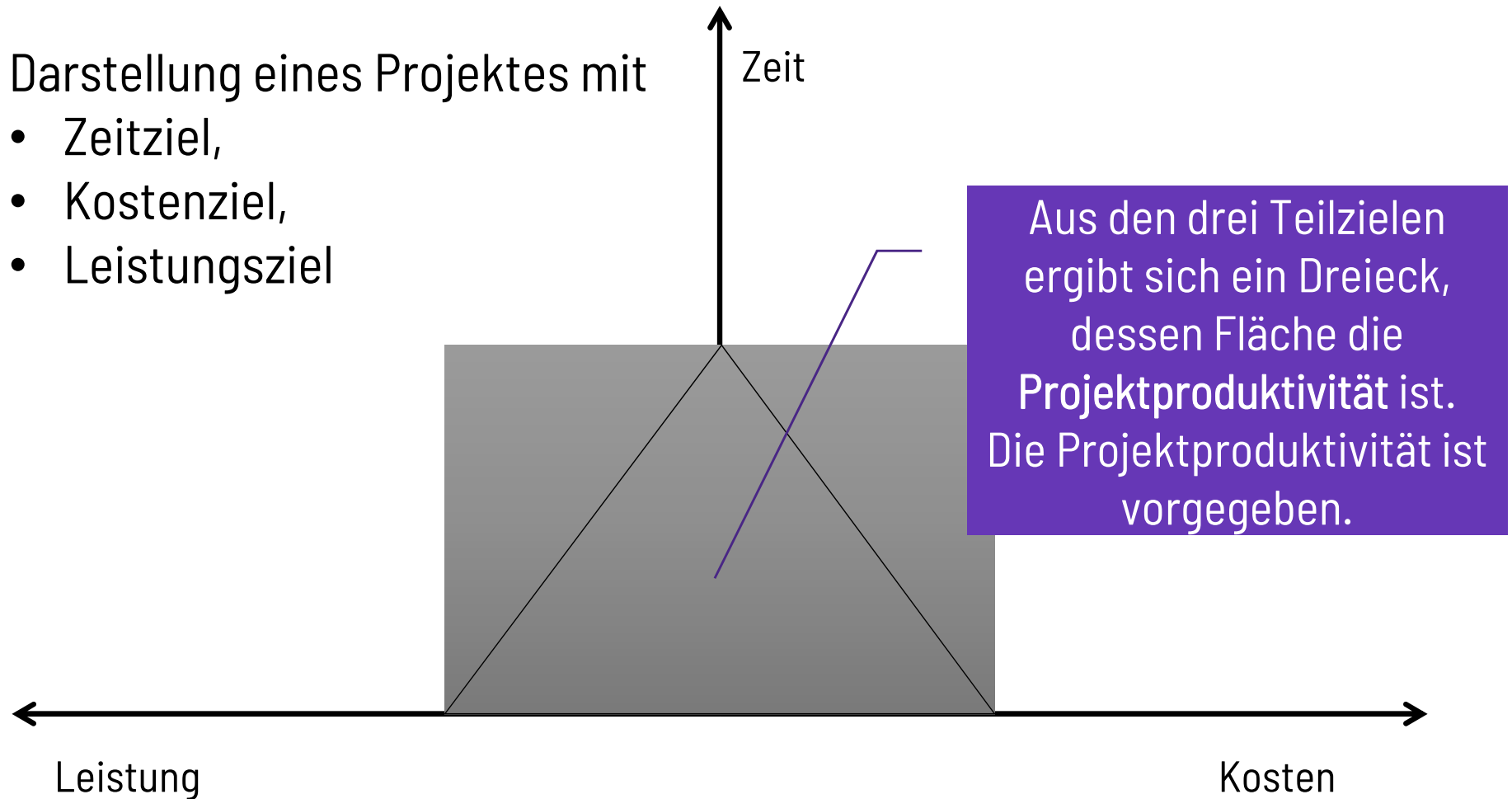
Eignung:

- Wenn Anforderungen klar definiert sind,
 - klare Prioritäten gesetzt werden können und
 - schrittweise Einführung einen Zeitvorteil gegenüber einer einmal erstellten „all singing all dancing“ – Version verspricht
- Mögli
 - könne
 - Realis

Exkurs: Das Projektdreieck

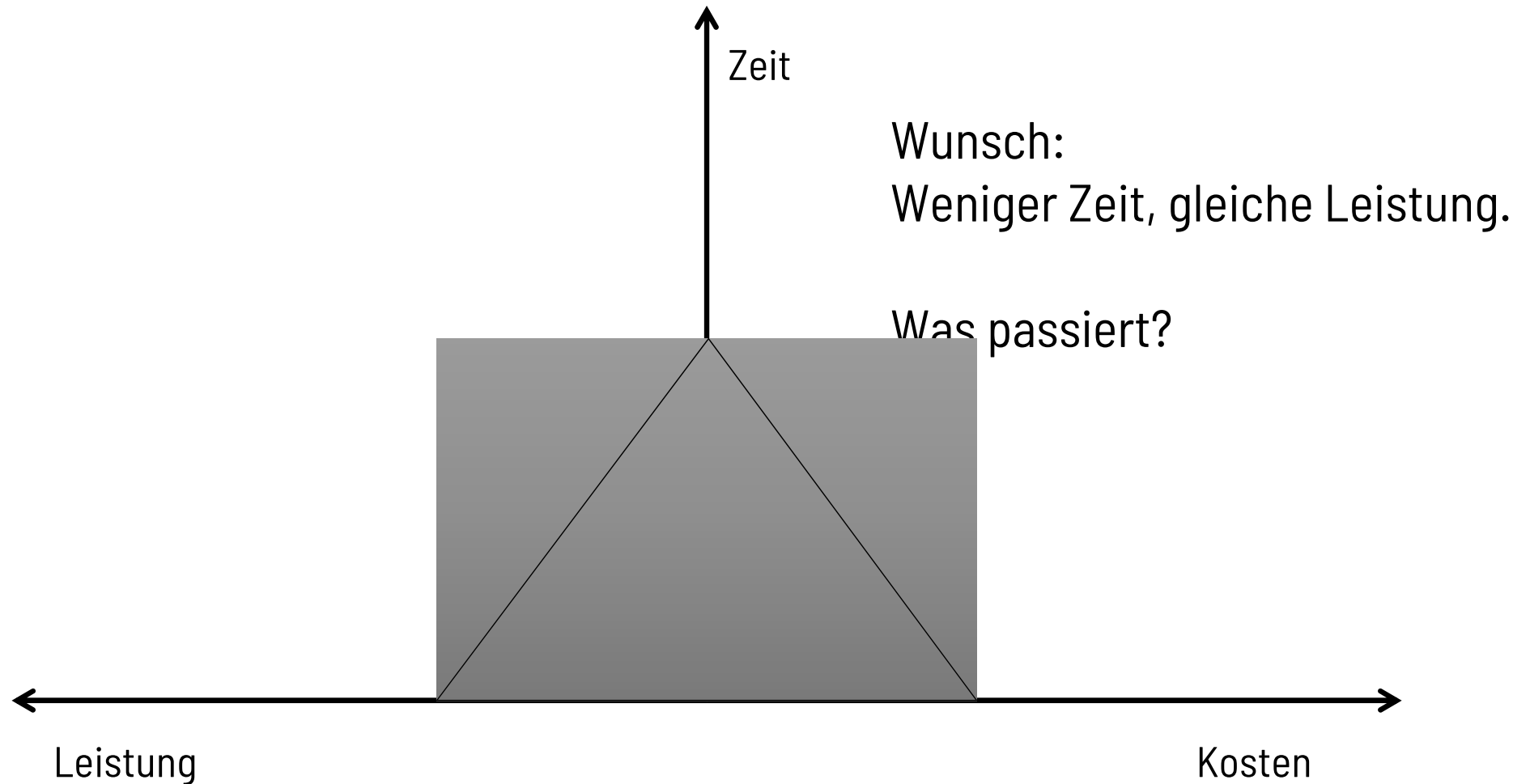
Darstellung eines Projektes mit

- Zeitziel,
- Kostenziel,
- Leistungsziel



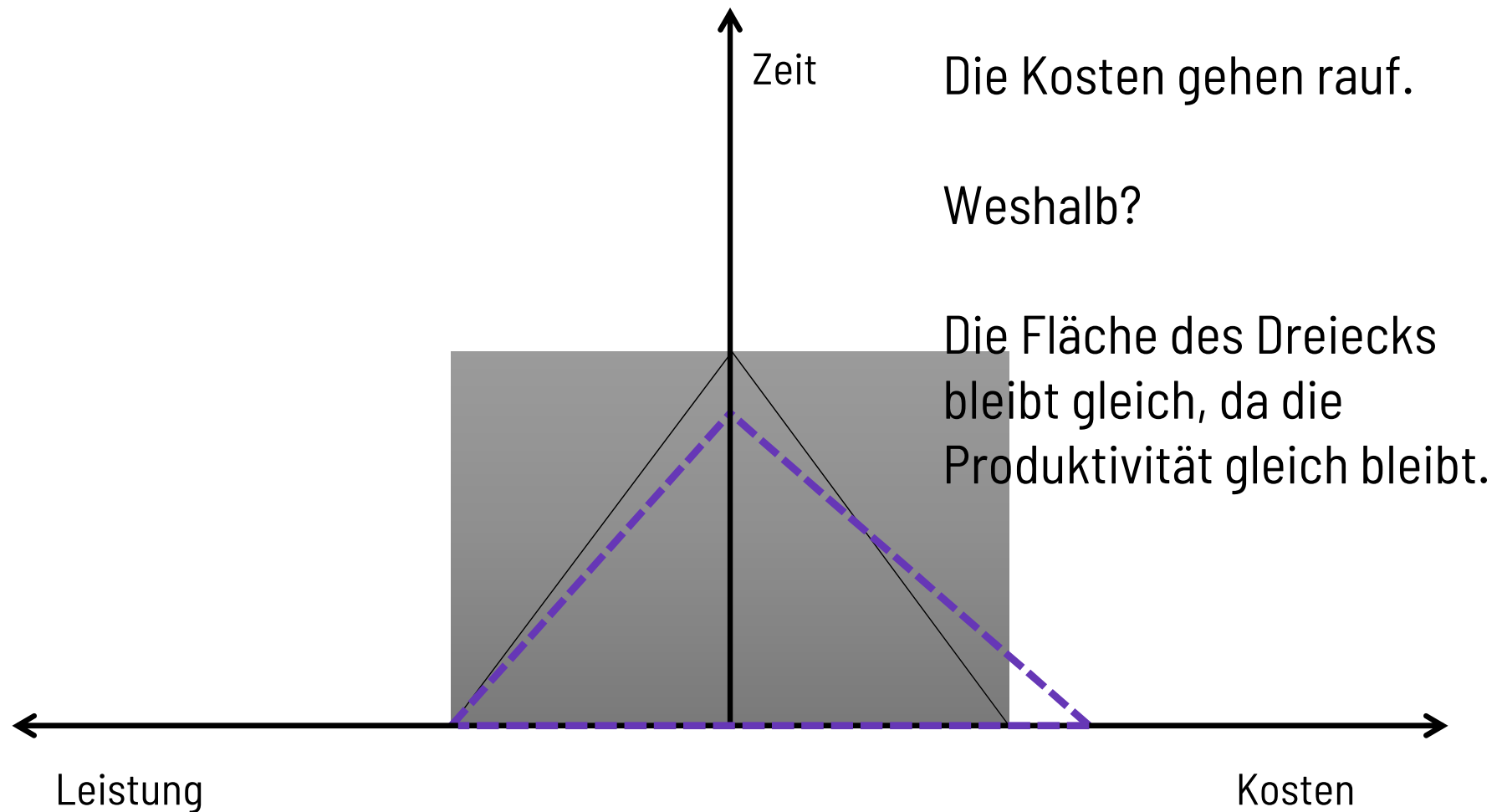
Die Pfeile zeigen immer in die für den Projektkunden ungünstige Richtung.

Exkurs: Das Projektdreieck



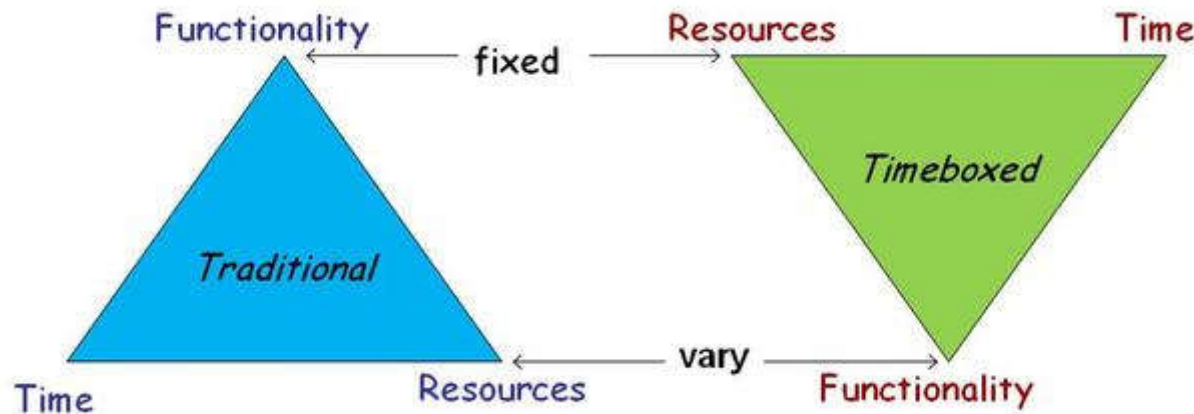
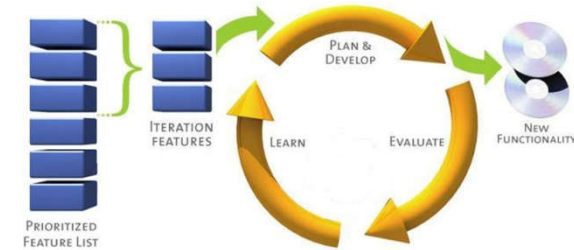
Die Pfeile zeigen immer in die für den Projektkunden ungünstige Richtung.

Exkurs: Das Projektdreieck



Die Pfeile zeigen immer in die für den Projektkunden ungünstige Richtung.

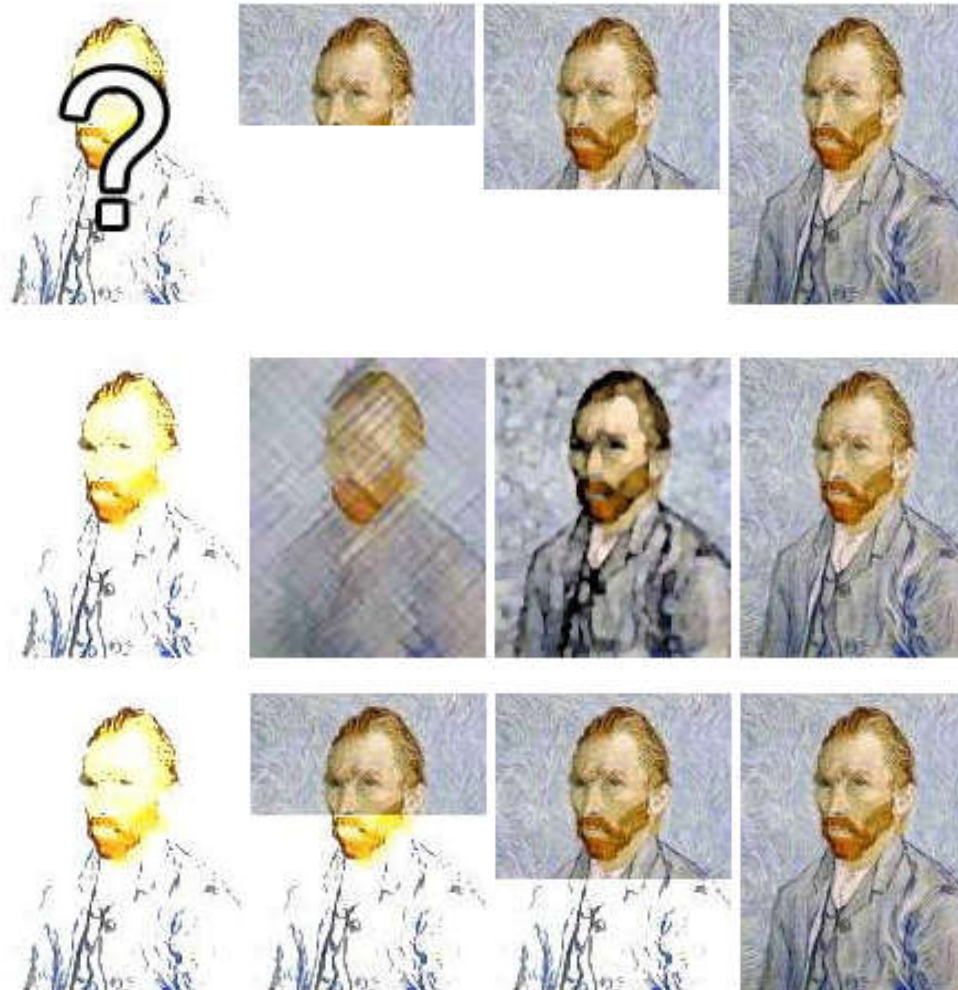
Inkrementelle Modelle: Eigenschaften - Erweiterungen



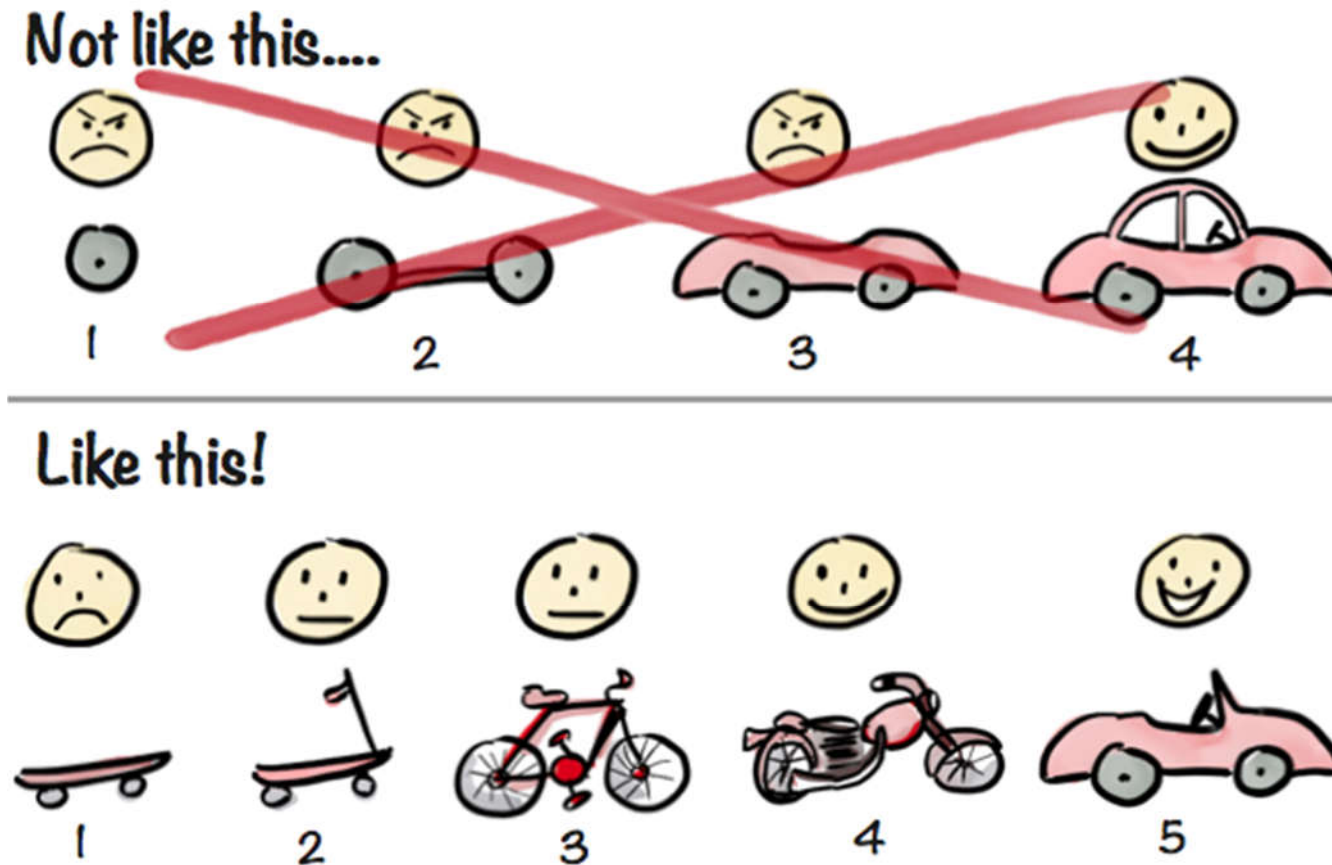
- Berücksichtigung von (neuen oder sich ändernden) Kundenanforderungen zum Beginn jedes Inkrements möglich
- Einführung von „Time-Boxed“-Iterationen erhöht Transparenz im kompletten Projektumfeld
 - Jedes Inkrement ist Ergebnis einer Iteration mit *immer* derselben Dauer (Time-Boxed) über die Entwicklungsdauer des Gesamtprodukts



Unterschiede zwischen iterativ und inkrementell



Unterschied zwischen „inkrementell“ und „iterativ-inkrementell“



Henrik Kniberg: <http://blog.crisp.se/2016/01/25/henrikkniberg/making-sense-of-mvp>

Beispiele für inkrementell-iterative Prozessmodelle

- Rational Unified Process (RUP)
 - Vgl. Buch von Tsui / Karam / Bernal
- allgemein sogenannte „agile“
Softwareprozessmodelle:
 - Extreme Programming XP
 - Scrum

„Am 11.-13. Februar, 2001 ...“



„... in Snowbird, Utah, USA.“

Manifest der Agilen Softwareentwicklung

„Ein Manifest (lat.: *manifestus* – *handgreiflich gemacht*) ist eine öffentliche Erklärung von Zielen und Absichten, oftmals politischer Natur.“

We are uncovering better ways of developing software by doing it and by helping others to do it.
We value:

- 1/ individuals and interactions over process and tools
- 2/ working software over comprehensive documents
- 3/ customer collaboration over contract negotiation
- 4/ responding to change over following a plan.

The Agile Alliance
Manifesto for Agile Software Development
Agile Process

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

von <http://www.agilemanifesto.org>

Exkurs: Prinzipien des „Agilen Manifests“



1. Our highest priority is to **satisfy the customer** through early and continuous delivery of **valuable software**.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. **Business people and developers must work together** daily throughout the project.

Exkurs: Prinzipien des „Agilen Manifests“



5. Build projects around **motivated individuals**. Give them the **environment** and **support** they need, and **trust** them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.
7. **Working software** is the primary measure of progress.
8. Agile processes promote **sustainable** development. The sponsors, developers, and users should be able to maintain a **constant pace** indefinitely.

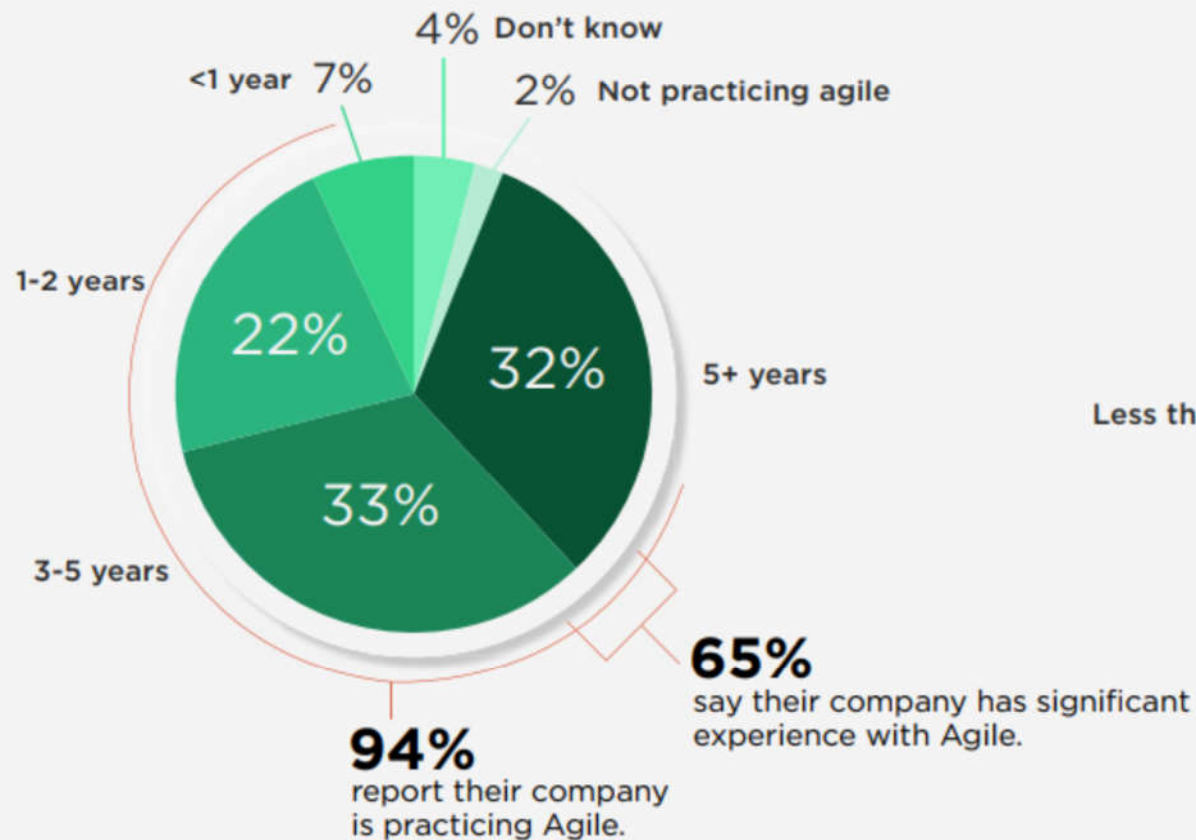
Exkurs: Prinzipien des „Agilen Manifests“



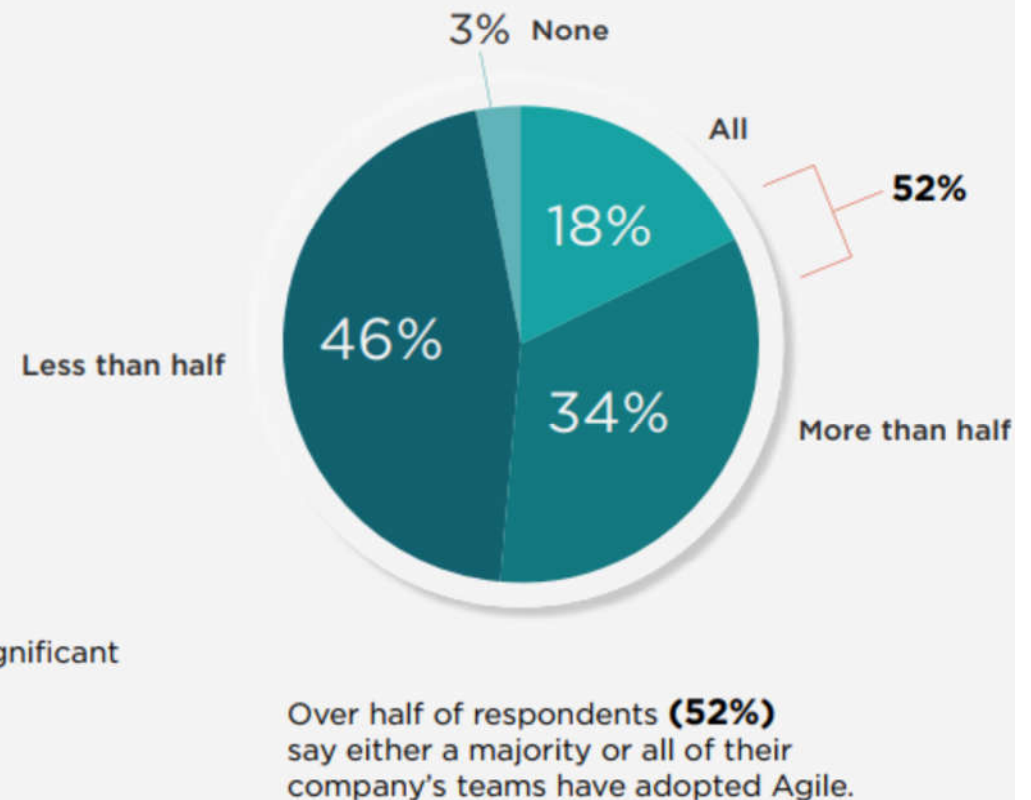
9. Continuous attention to **technical excellence** and good design enhances agility.
10. **Simplicity** – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
12. At regular intervals, the **team reflects** on how to become more effective, then **tunes** and **adjusts** its behavior accordingly.

Verbreitung agiler Methoden 2021

How long has your company been practicing agile?

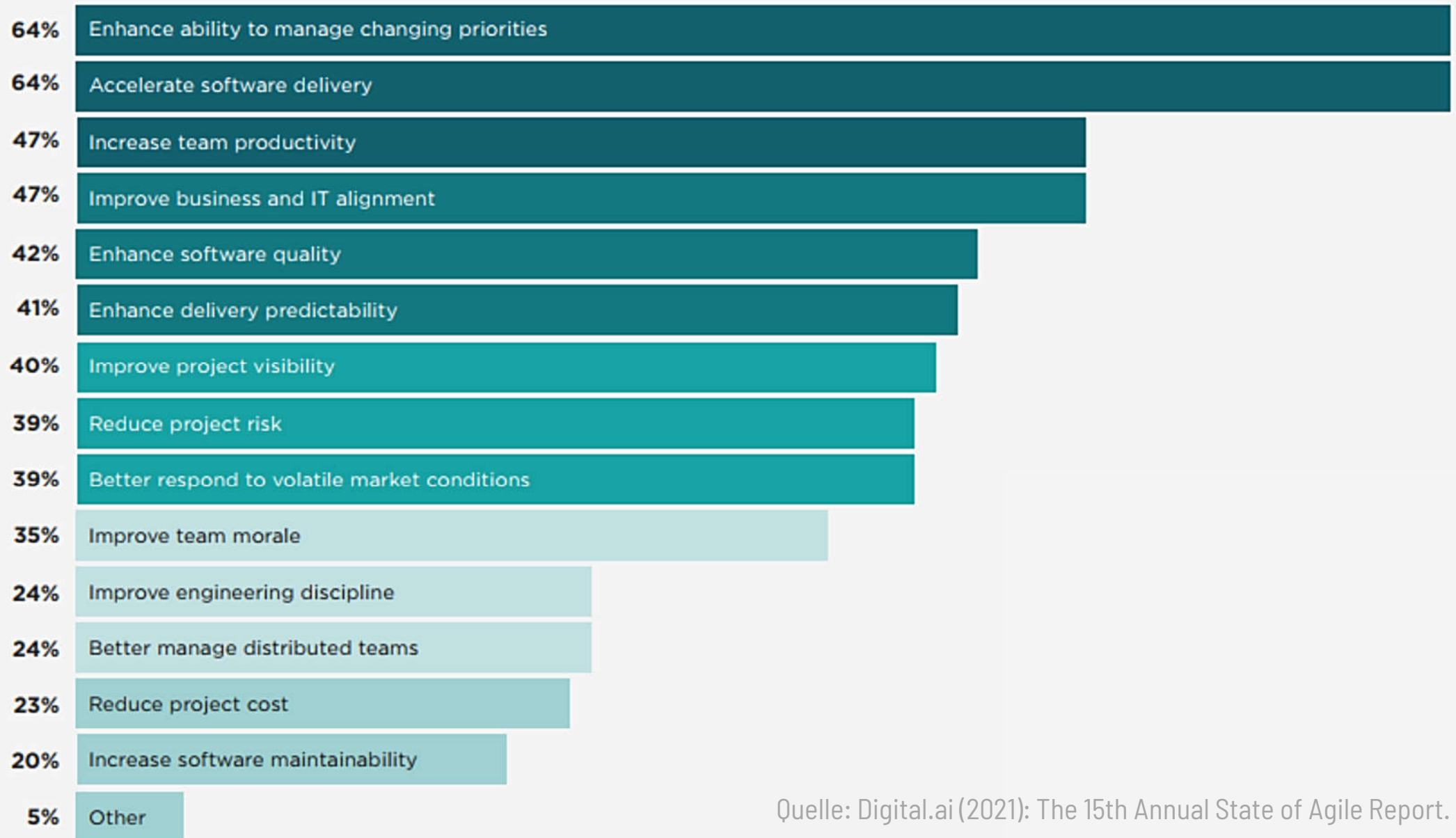


How many of your company's teams have adopted agile practices?



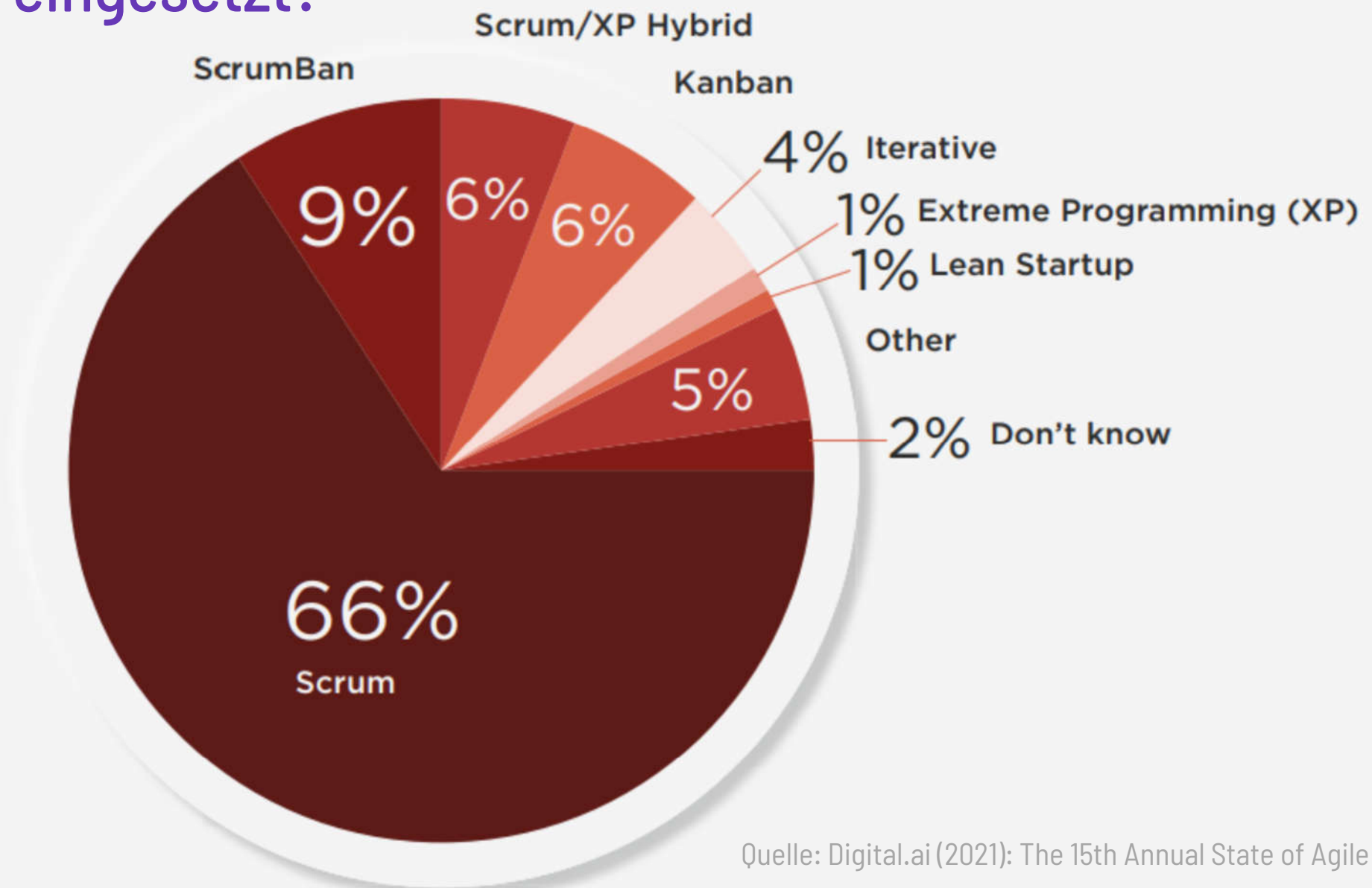
Quelle: Digital.ai (2021): The 15th Annual State of Agile Report.

Gründe für den Einsatz agiler Methoden 2021



Quelle: Digital.ai (2021): The 15th Annual State of Agile Report.

Welche agile Methode werden weltweit 2021 eingesetzt?



Quelle: Digital.ai (2021): The 15th Annual State of Agile Report.

Auch andere
Aspekte der
Vorgehensweise
sind wichtig!

Ranking von
Erfolgsfaktoren der
Standish Group aus 21
CHAOS-Reports bis 2015

FACTORS OF SUCCESS	POINTS
Executive Sponsorship	15
Emotional Maturity	15
User Involvement	15
Optimization	15
Skilled Resources	10
Standard Architecture	8
Agile Process	7
Modest Execution	6
Project Management Expertise	5
Clear Business Objectives	4



Scrum



© Jolon Penna, CC BY-SA 2.0

Was ist Scrum?

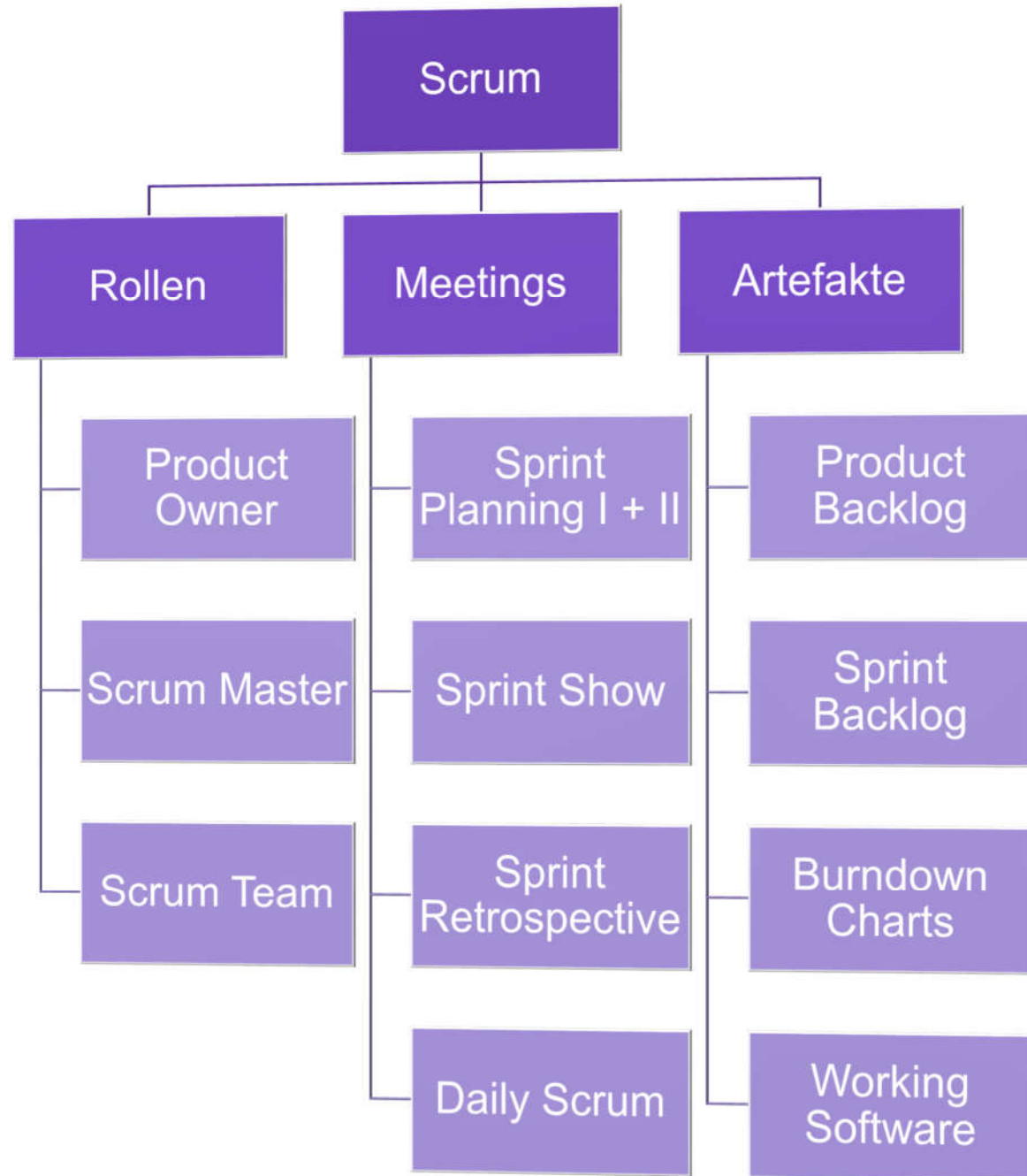


- Empirisches Softwareprozess-Modell
 - Transparenz
 - Überprüfung
 - Anpassung
- Iterativ-inkrementelles Vorgehensmodell
 - „Done“-Inkremente
 - Time-Boxed
- Fokus auf dem Erreichen **geschäftsrelevanter Ziele**
- Sammlung von **einfachen Arbeitstechniken**, Strukturen, Rollen und Methoden
- **Selbst-organisierende Teams**, Feature-Orientierte Besetzung der Teams

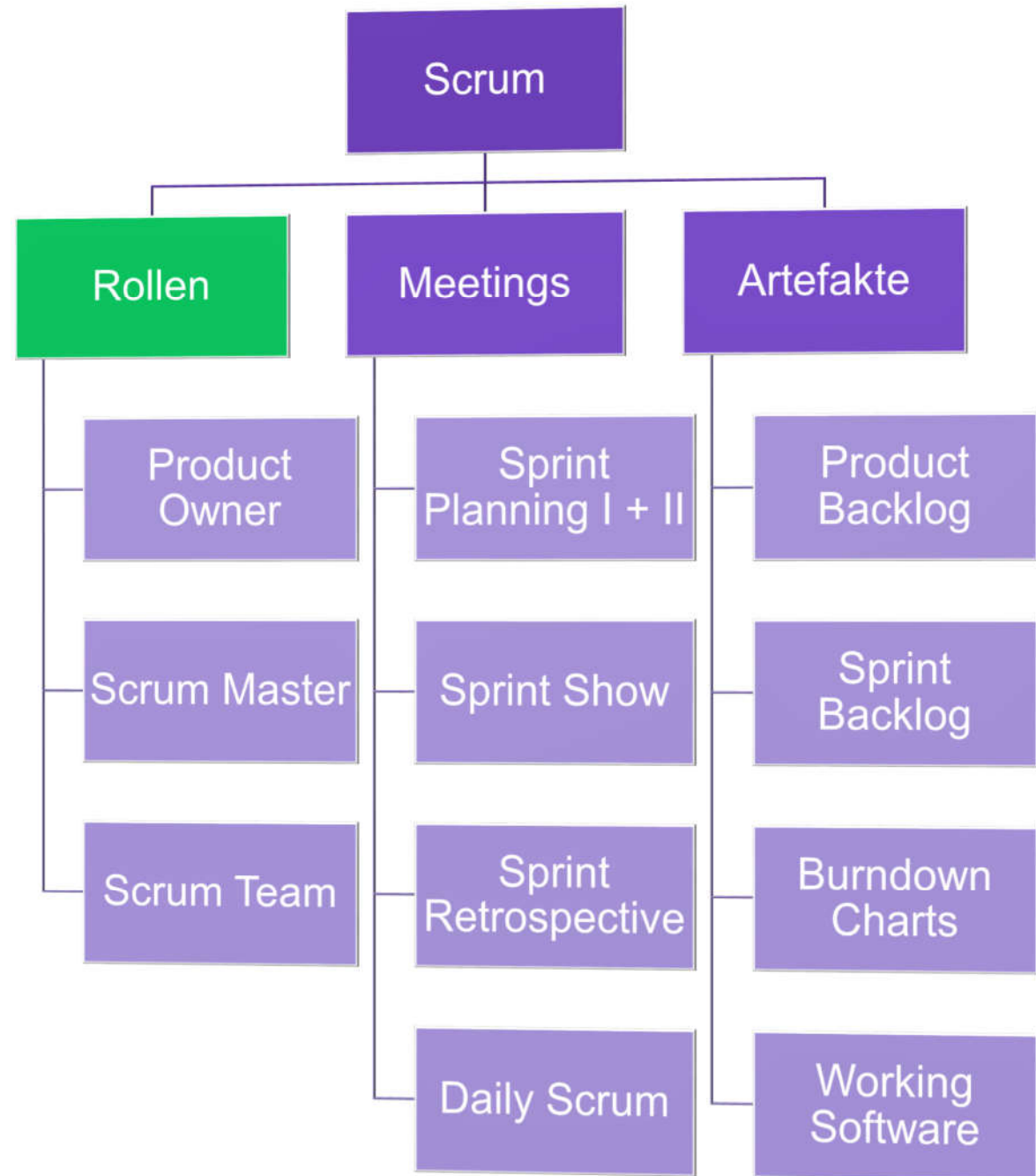
Scrum – wichtige Eigenschaften

- Regelmäßige Lieferung produktreifer Software in kurzen Abständen (etwa monatlich)
- Ausgeprägtes Bewusstsein für Qualitätsarbeit
- Kontinuierliche Transparenz über aktuellen Projektfortschritt
- Keine Flaschenhälse durch schnelle und fortwährende Rückmeldungszyklen
- Große Anpassungsfähigkeit an neue Anforderungen
- Klare Entscheidungswege und Verantwortlichkeiten

Scrum – schematischer Überblick



Scrum – Rollen



Scrum – Rollen

- Betreibt Stakeholder Management
- Definiert Anforderungen
- Verantwortlich für Wertschöpfung des Projekts
- Legt alle Priorisierungen fest
- Annahme oder Ablehnung der Projektergebnisse



Product Owner



Scrum Master

Scrum Team

- Außenminister für das Team
- Repräsentiert das Management
- Coach
- Eliminiert Hindernisse
- Schützt vor externen Störgrößen



- Selbstorganisierendes Team
- Große Autonomie
- Undifferenzierte Rollen
- Typischerweise 6-9 Personen



Product Owner

am Beispiel einer Stellenanzeige

Produktmanager (m/w)

(Kennziffer: 2012005128)

Ihre Aufgaben

Sie ist schon heute Realität: Patientenzentrierte Medizin, die durch intelligente Software unterstützt wird. Die sogenannte Software Assisted Medicine gibt dem Arzt bei anstehenden Entscheidungen patientenindividuelle Informationen – genau dann, wenn er sie benötigt. Sie sind als Produktmanager SAM dafür zuständig, dass der Ansatz der Software Assisted Medicine (SAM) in bestehenden und neuen Produkten unseres Mandanten Einzug und weiteren Ausbau findet. Sie identifizieren und analysieren Anwendungsbereiche (z.B. Medizinische Behandlungspfade). Sie erstellen Anforderungen an die Entwicklung und sorgen für deren Umsetzung. Als Grundlage dient eine umfassende Patientendatenbank auf deren Basis SAM arbeitet. Ihre Software hilft dem behandelnden Arzt bessere Entscheidungen zu treffen. Basierend auf den Zielen und der Philosophie unseres Mandanten entwickeln Sie Konzepte zur Weiterentwicklung der bestehenden Software sowie zur Steigerung der Effizienz und zur Verbesserung der Margen.

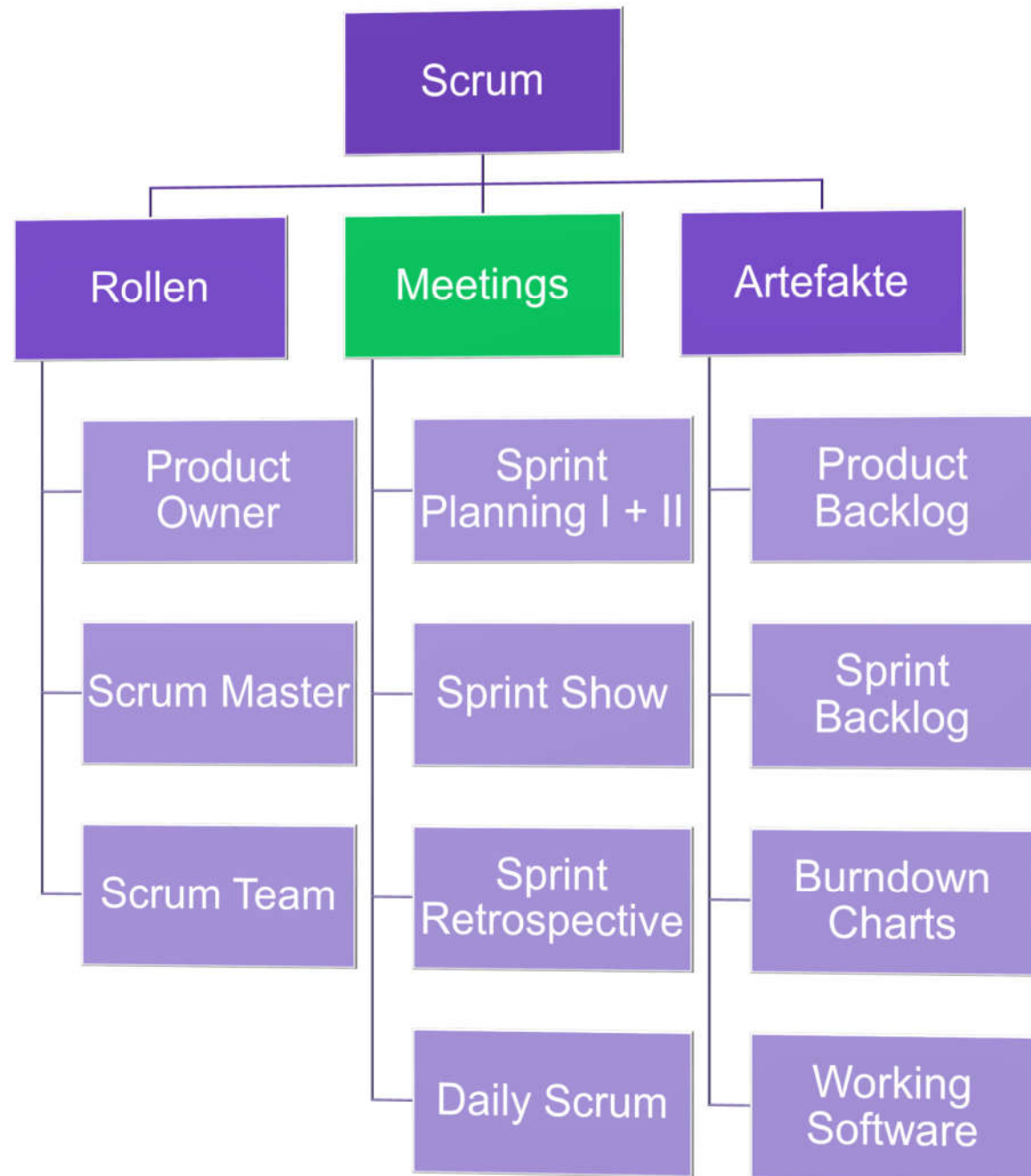
- Verantwortung der Produktstrategie, -konzeption und -spezifikation hinsichtlich Markt, Technologie, Qualität
- Kunden-Anforderungen aufnehmen, priorisieren und für das Entwicklungsteam aufbereiten
- Analyse bestehender Geschäftsstrukturen, -prozesse und -systeme im Hinblick auf Potenzialoptimierung und Effizienzsteigerung
- Entwicklung von individuell angepassten und nachhaltigen Konzepten zur Hebung von Effizienz-Reserven und Margen-Verbesserungen
- Vermittlung und Verankerung von Erfolgskonzepten zur Sicherung der Nachhaltigkeit
- Sicherung der Nachhaltigkeit durch Erfolgskontrolle
- Erstellung von regelmäßigen Reports für das Management Board
- Mitarbeit im agilen Softwareentwicklungsprozess als Product-Owner

Ihr Profil

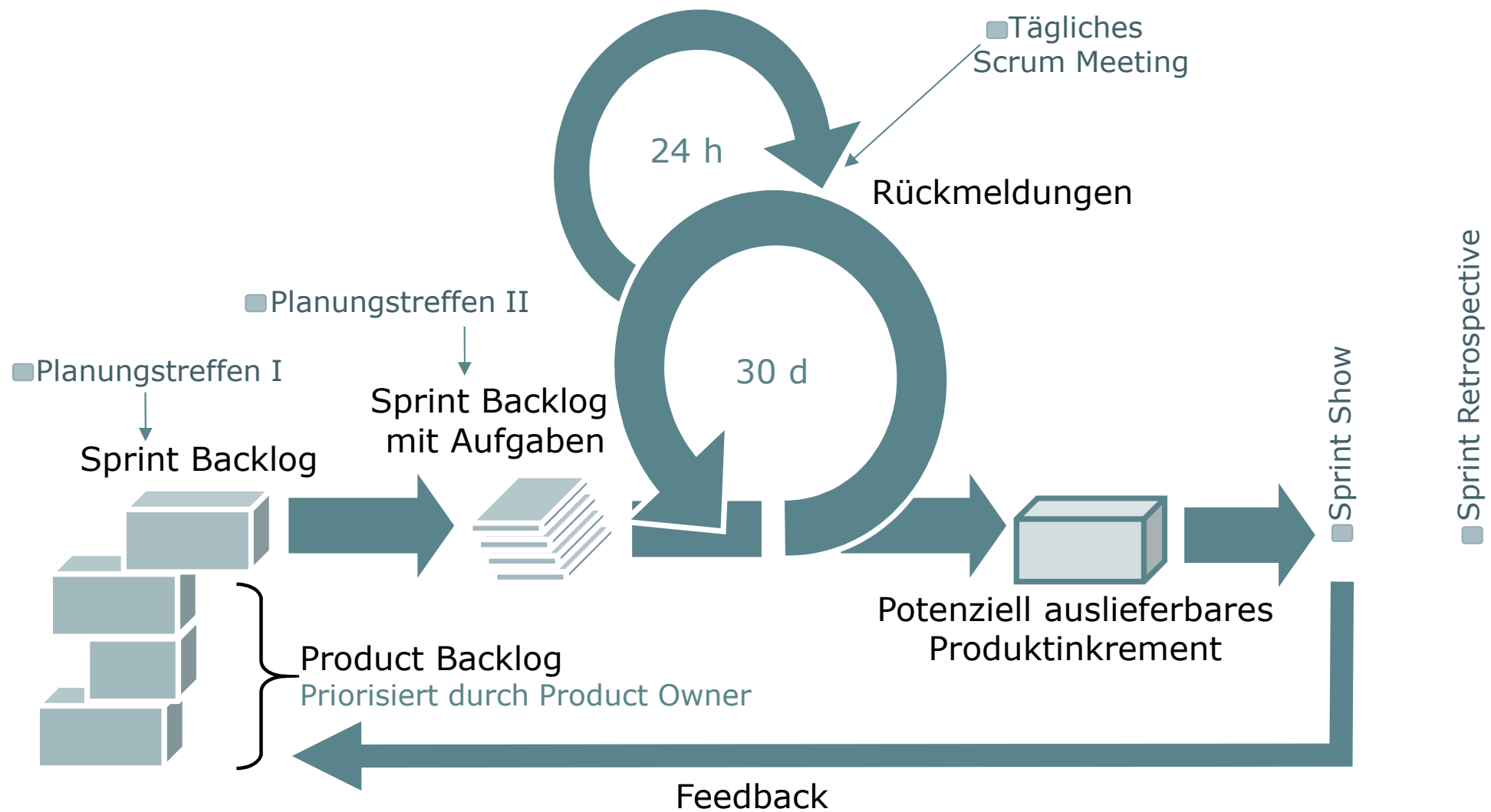
- Erfahrung in der Anforderungsanalyse und Konzeption komplexer Softwaresysteme
- Sie haben ein IT-Studium oder vergleichbare Ausbildung erfolgreich abgeschlossen
- Sie verfügen über Erfahrungen und nachweisbare Erfolge in Prozessen der Geschäftsentwicklung
- Sie besitzen ausgezeichnetes finanzielles Verständnis, analytisches Gespür wie auch konzeptionelle Stärke
- Sie besitzen die Fähigkeit, Menschen für ihre Ideen zu gewinnen und diese kooperativ einzubinden
- Ausgezeichnete mündliche und schriftliche Ausdrucksweise in deutscher und englischer Sprache
- Wünschenswert sind erste Erfahrungen im Produktmanagement von Softwareprodukten
- Analytisches Denken und konzeptionelle Stärke
- Durchsetzungsvermögen
- Ergebnisorientierung
- Selbstständige Arbeitsweise
- Teamfähigkeit und Belastbarkeit

Wenn Sie bereit sind, Verantwortung zu übernehmen, höchstes Engagement und Offenheit für Veränderung mitzubringen um gemeinsam mit uns kreative Lösungen zu entwerfen, stehen Ihnen interessante Herausforderungen

Scrum – Meetings



Scrum – Abläufe – ein prozessorientierter Überblick



Sprint Planing 1 und 2

- Ziele
 - *Was* werden wir im nächsten Sprint realisieren?
 - *Wie* wird das Entwicklungsteam es realisieren?
- Teilnehmer
 - Sprint Planning 1: Product Owner, Team, Scrum Master
 - Sprint Planning 2: Team, Scrum Master; Product Owner stand-by
- Dauer 2h-8h
- Abschätzung des Teams
- Versprechen des Teams

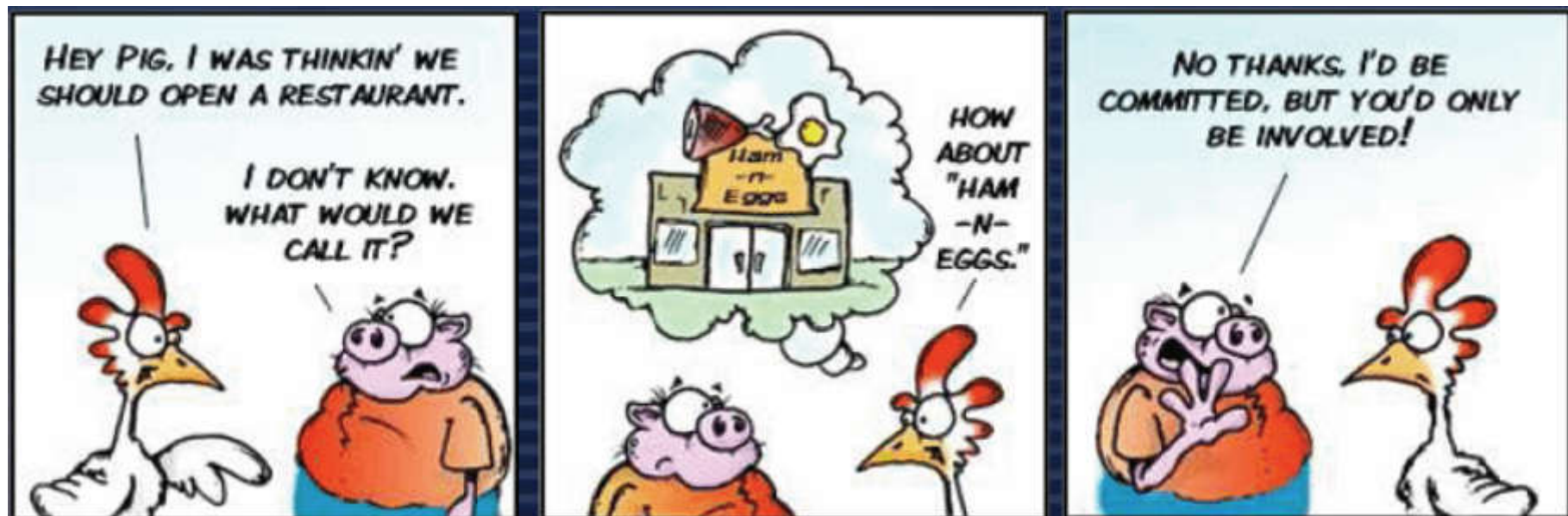
Sprint Show

- **Ziele**
 - Demonstration der Features, die im vergangenen Sprint erstellt wurden
 - Finale Abnahme der Features durch den Product Owner
- **Teilnehmer:** Alle(!) Stakeholder
- **Dauer:** Ca. 15min – 30min

Sprint Retrospective



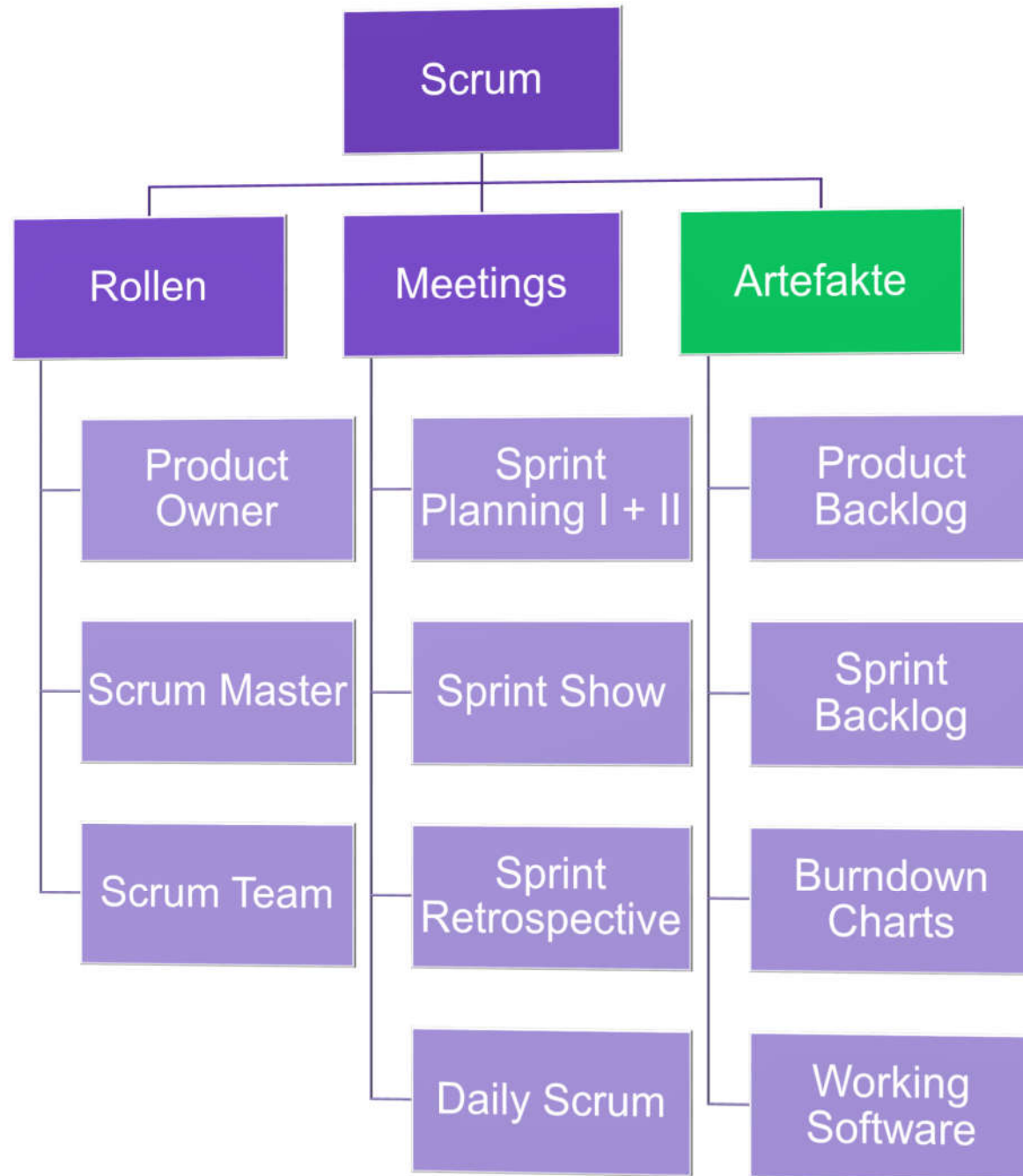
Daily Scrum - Pigs & Chickens



Daily Scrum – „3 W-Fragen“



Scrum – Artefakte



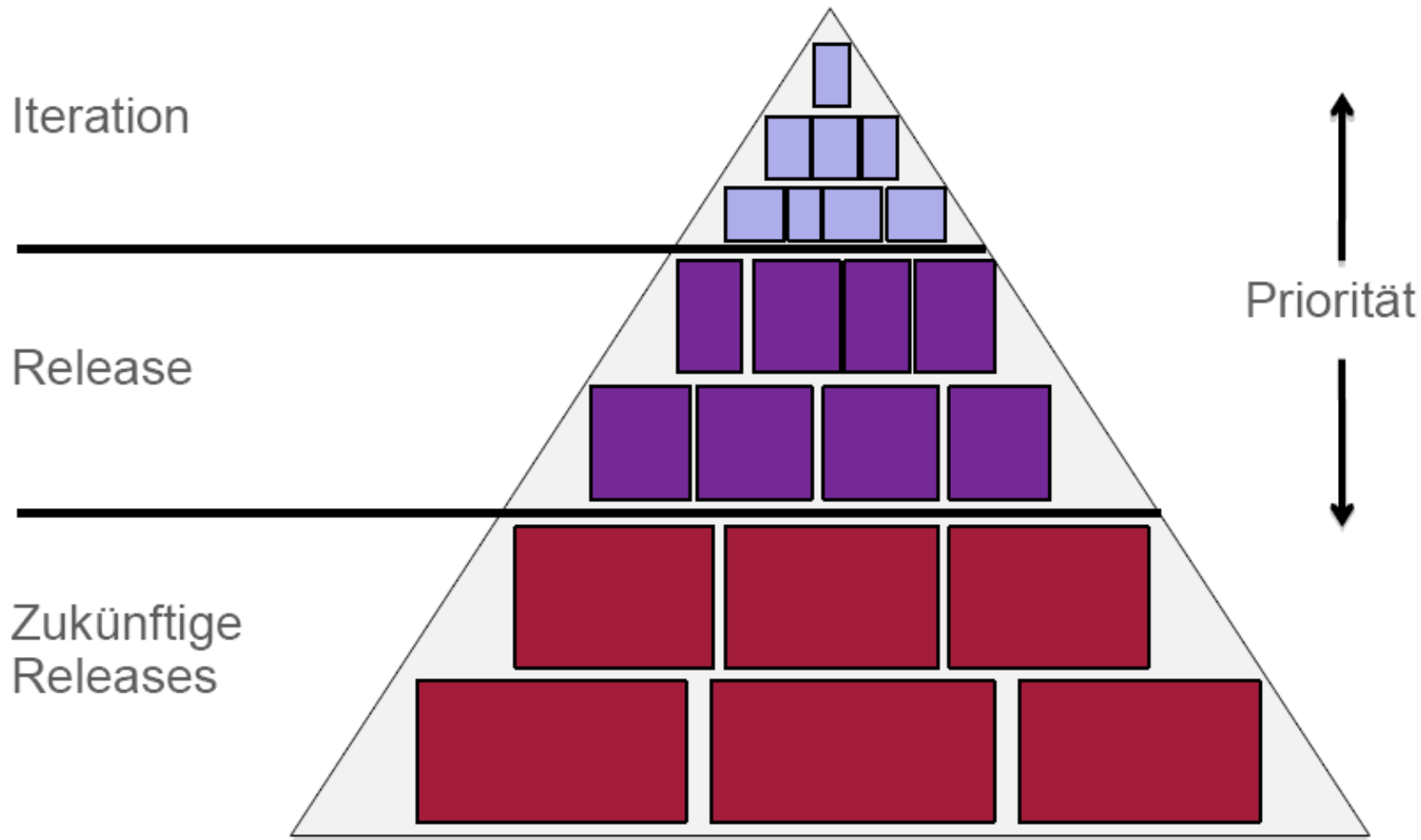
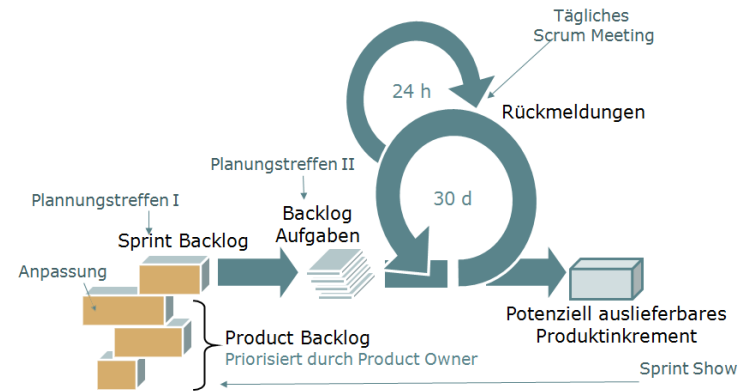
Scrum Backlogs

- **Backlog:** Rückstand, Arbeitsrückstand, Auftragsbestand
- **Scrum Backlogs:** Instrumente zur Strukturierung und Planung von Aufgaben in Scrum.

Es gibt zwei verschiedene:

- **Product Backlog**
- **Sprint Backlog**

Product Backlog



Product Backlog

ID	Story	Estimation	Priority
7	As an unauthorized user I want to create a new Account	3	1
1	As an unauthorized user I want to login	1	2
10	As an authorized User I want to logout	1	3
9	Create script to purge database	1	4
2	As an authorized User I want to see the list of items so that I can select one	2	5
4	As an authorized User want to add a new item so that it appears In the list	5	6
3	As an authorized User I want to delete the selected item	2	7
5	As an authorized User I want to edit the selected item	5	8
6	As an authorized User I want to set a reminder for a selected item so that am reminded when item is due	8	9
8	As an administrator I want to see the list of accounts on login	2	10

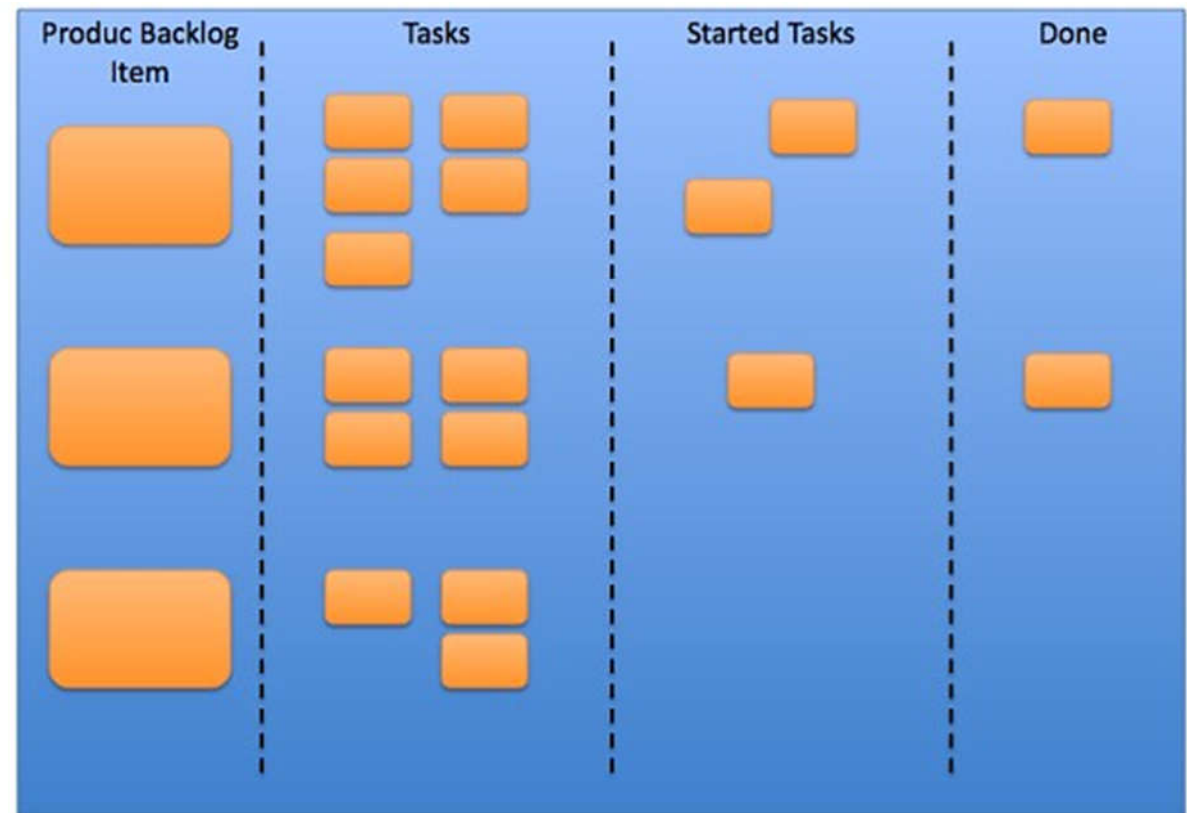
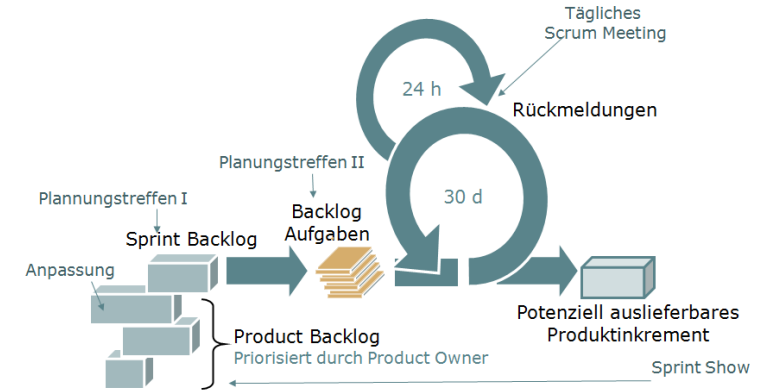
Product Backlog

Backlog Description	Initial Estimate	Adjustment Factor	Adjusted Estimate	1 week remaining until completion						
				1	2	3	4	5	6	7
Title Import				253	269	139	140	140	140	140
Project selection or new	5	0.2	3.6	3.6	0	0	0	0	0	0
Template backlog for new projects	2	0.2	3.4	2.4	0	0	0	0	0	0
Create product backlog worksheet with formatting	3	0.2	3.5	3.5	0	0	0	0	0	0
Create sprint backlog worksheet with formatting	3	0.2	3.6	3.6	0	0	0	0	0	0
Display tree view of product backlog, releases, sprints	2	0.2	2.4	2.4	0	0	0	0	0	0
Sprint-1	13	0.2	15.6	16	0	0	0	0	0	0
Create a new window containing product backlog template	5	0.2	3.6	3.6	0	0	0	0	0	0
Create a new window containing sprint backlog template	2	0.2	3.4	2.4	2.4	0	0	0	0	0
Burndown window of product backlog	5	0.2	6	6	6	0	0	0	0	0
Burndown window of sprint backlog	1	0.2	1.2	1.2	1.2	0	0	0	0	0
Display tree view of product backlog, releases, sprints	2	0.2	2.4	2.4	2.4	0	0	0	0	0
Display burndown for selected sprint or release	5	0.2	3.6	3.6	3.6	0	0	0	0	0
Sprint-2	18	0.2	19.2	19	19	1.2	0	0	0	0

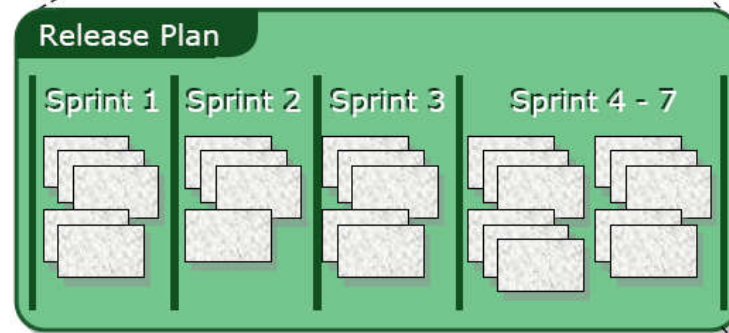
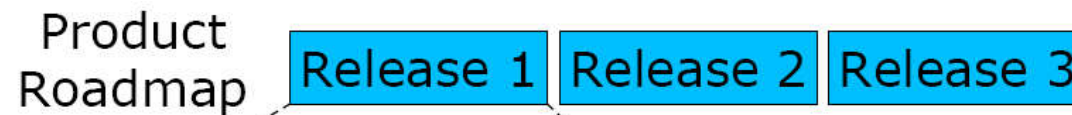
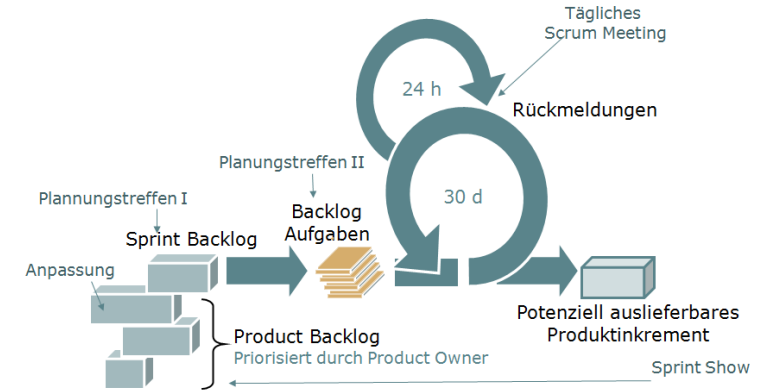
- Priorisierte Liste von Anforderungen oder Eigenschaften, „Living Document“
- **Attribute:** ID, Kurzbeschreibung, Wichtigkeit (Priorisierung), Initiale Schätzung, High-Level-Beschreibung, Kommentare
- **Zusätzlich:** Anfrager (Requestor), (techn.) Komponenten, adaptierte Schätzung
- **Product Owner:** Manager des Product Backlogs
 - Inhalt, Verfügbarkeit, Priorisierung

Sprint Backlog

- **Priorisierte Untermenge** von Einträgen des Product Backlogs, die für einen Sprint ausgewählt wurde, **plus Plan zur Lieferung** des Produktinkrements
- Also: **Priorisierte Liste technischer Aufgaben** für das Team in einem Sprint
- Damit: Vorhersage des Scrum-Teams, welche Funktionalität im nächsten Produktinkrement enthalten sein wird und welche Arbeit zu dessen vollständiger Lieferung nötig ist
- Typischer Aufwand pro Aufgabe: 2h-16h



Releases und Sprint Backlog



Sprint Backlog

Task 1	8
Task 2	6
Task 3	12
Task 4	5
Task 5	4

Copyright (c) 2006-2007, Innolution, LLC and Mountain Goat Software, Inc. All Rights Reserved.

Backlogs in Rational Team Concert (RTC)

Releaserückstand ▾

Owner: Mensabewertungsportal | Iteration: Release 1.0 | [10 Closed](#) | [14 Open](#) | Progress: 39/93 Pkte. | Estimated: 100%

Release 1.0
Closed items: 10 | Open items: 14 | Progress: 39/93 Pkte. | Estimated: 100%

Maximum an Bewertungen abgeben	0 Punkte	--	Mittel	1042
Bewerten von Mahlzeiten	0 Punkte	--	Mittel	733
Durchschnitt von Bewertungen	0 Punkte	--	Mittel	727
Änderungen am Menüplan vornehmen	0 Punkte	--	Niedrig	756
Monatliche Bewertungen	0 Punkte	--	Niedrig	736
Möglichkeit Kommentar abzugeben	0 Punkte	--	Niedrig	731
2. Sprache Englisch	0 Punkte	--	Niedrig	729

Sprint 1 (1.0)
Closed items: 7 | Open items: 0 | Progress: 31/31 Pkte. | Estimated: --

Initial import	0 Punkte	--	Hoch	741
Cloud-Beispiel	0 Punkte	--	Hoch	876
Bewertung abgeben	5 Punkte	--	Hoch	768
Tagesplan anzeigen	3 Punkte	--	Hoch	766
Grundlagen	0 Punkte	--	Hoch	904
Cloud-Anbindung	20 Punkte	--	Hoch	740
Batch-Import (1 Tag)	3 Punkte	--	Mittel	747

Sprint 2 (1.0)
Closed items: 0 | Open items: 2 | Progress: 0/14 Pkte. | Estimated: 100%

Sprint 2 (1.0)
Closed items: 1 | Open items: 3 | Progress: 8/15 Pkte. | Estimated: 100%

Sprint 3 (1.0)
Closed items: 0 | Open items: 4 | Progress: 0/33 Pkte. | Estimated: 100%

Layout und Styling optimieren	20 Punkte	--	Hoch	1046
-------------------------------	-----------	----	------	------

Overview | **Planned Items** | Charts

View As
☒ Iterations
☐ Ranked List
☐ Teams
☐ Work Breakdown
[Edit](#) | [Copy](#)

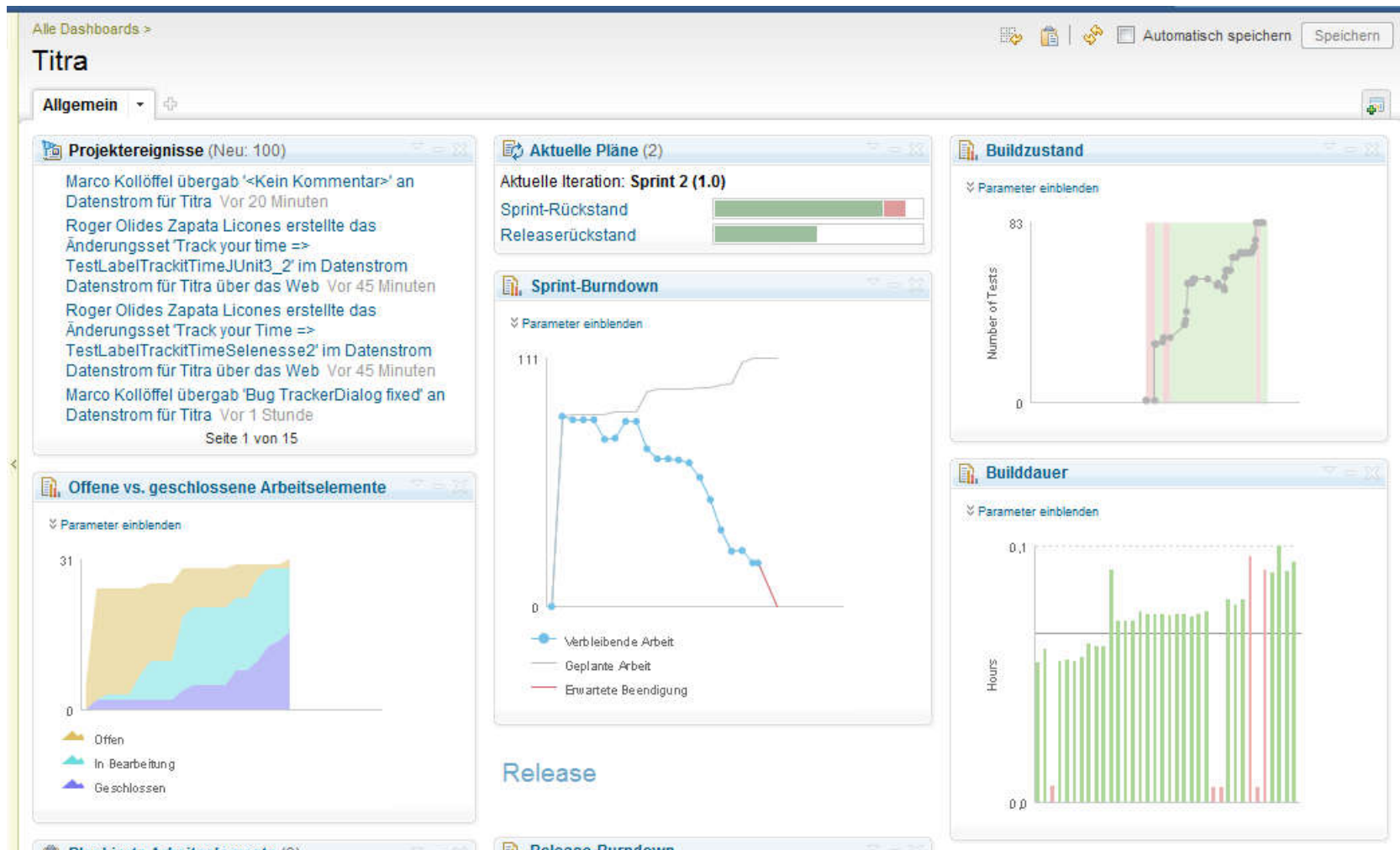
Actions
[Re-sort](#)

Exclude
☐ Assigned Items
☐ Empty Groups
☐ Estimated Items
☐ Execution Items
☐ Items from Sub-Iterati...
☐ Items from Sub-Teams
☒ Planned Items
☐ Planned Items
☐ Unchanged Items

Related Plans
[Produktrückstand \[Rückstand\]](#)

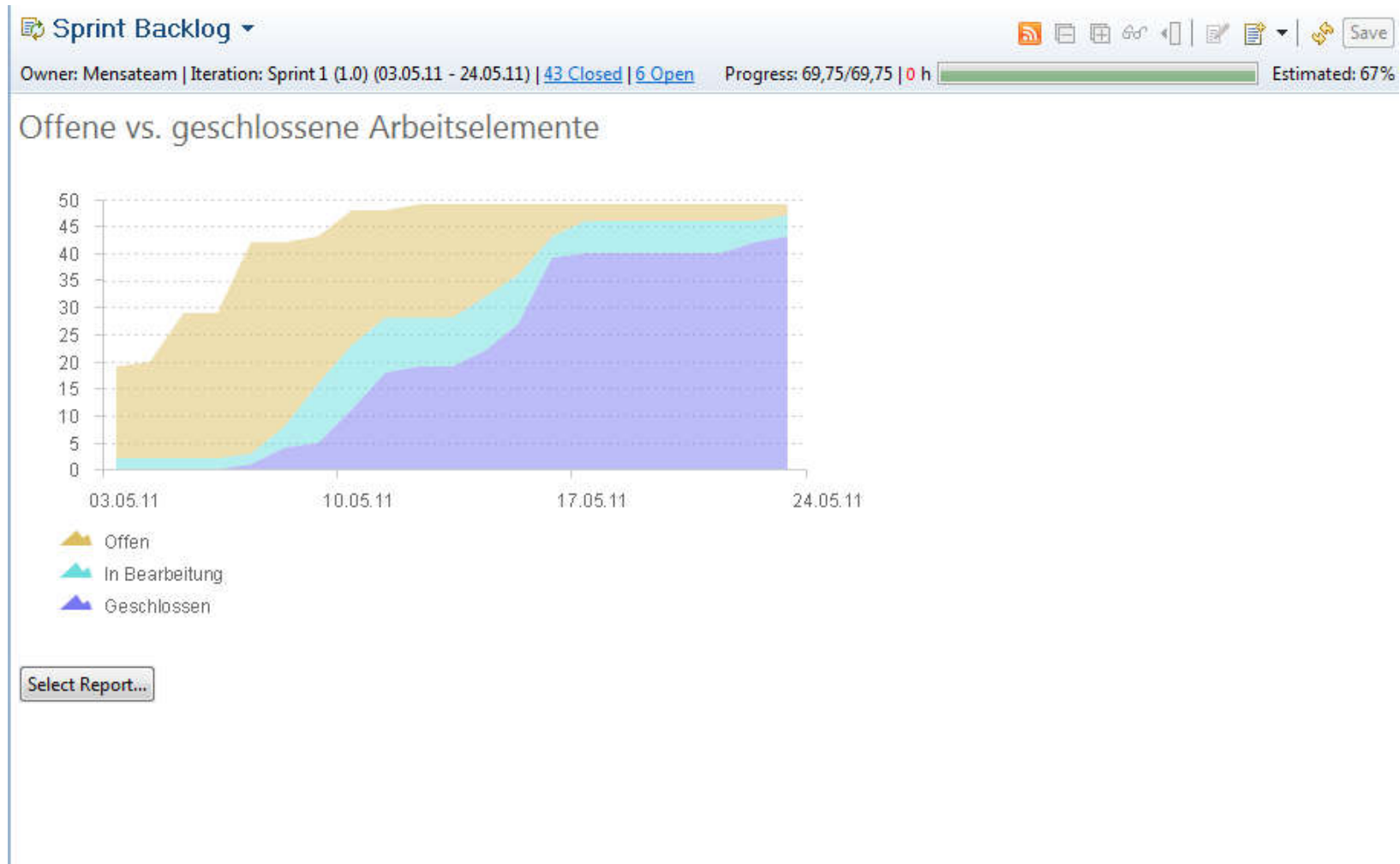
Zusätzliche Steuerungsinstrumente bei Scrum

Burndown-Chart



Zusätzliche Steuerungsinstrumente bei Scrum

Weitere Steuerungsgrafiken



Zusätzliche Steuerungsinstrumente bei Scrum

Definition of Done (DoD)

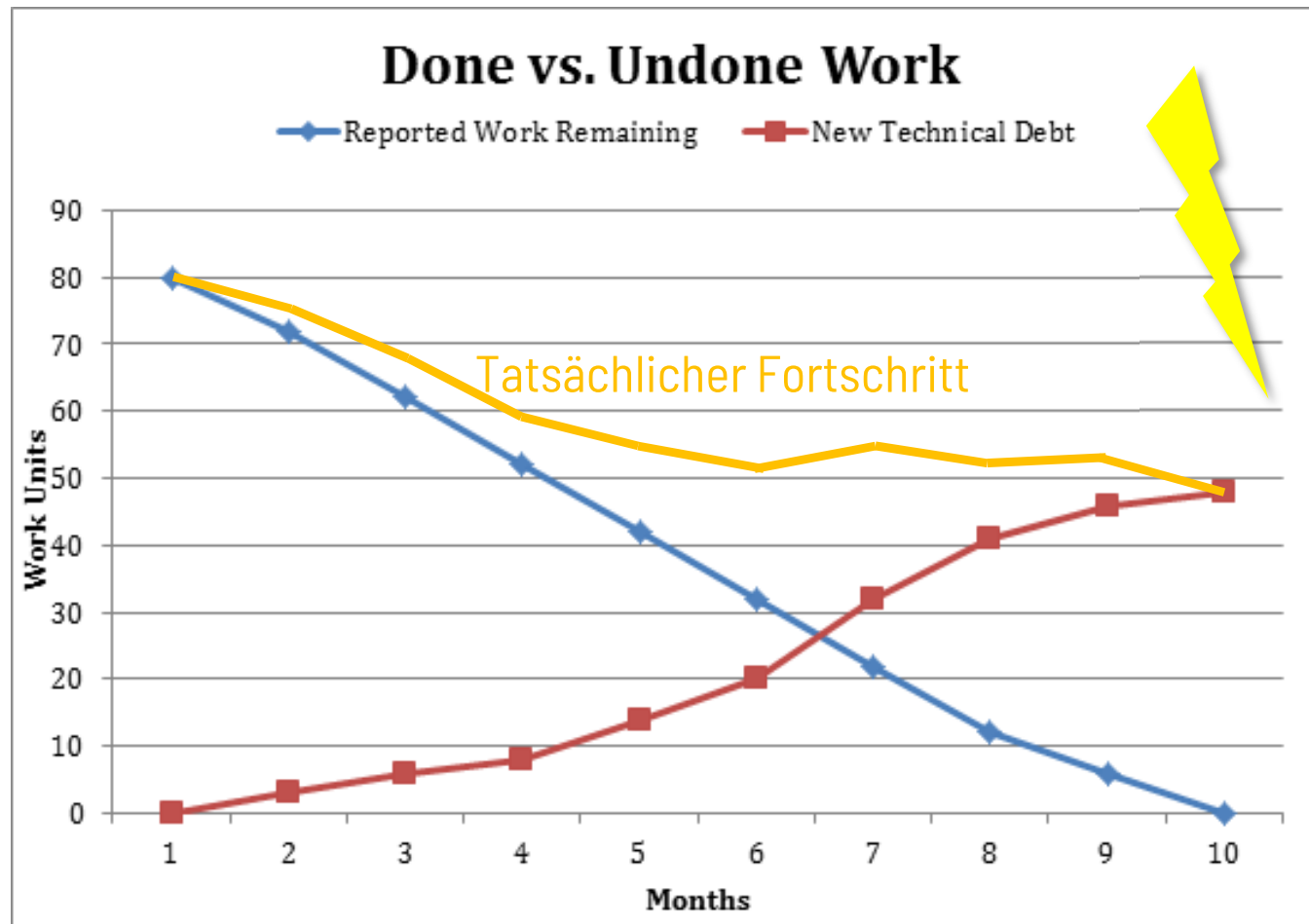
Was heißt denn „Done“ konkret?

Beispiel:

- Alle Tests entwickelt? Durchgeführt? Bestanden?
- Dokumentation erweitert? Release Notes?
- Codekonventionen eingehalten?
- Integration mit anderen Arbeiten durchgeführt?
- ...

➤ Eindeutige und einheitliche Festlegung, was „Done“ heißt, nötig.

Zusätzliche Steuerungsinstrumente bei Scrum Beispiel ohne Einsatz von „DoD“...



Das Management denkt:
Fertig! Ausliefern!

Leider „technische
Schulden“:

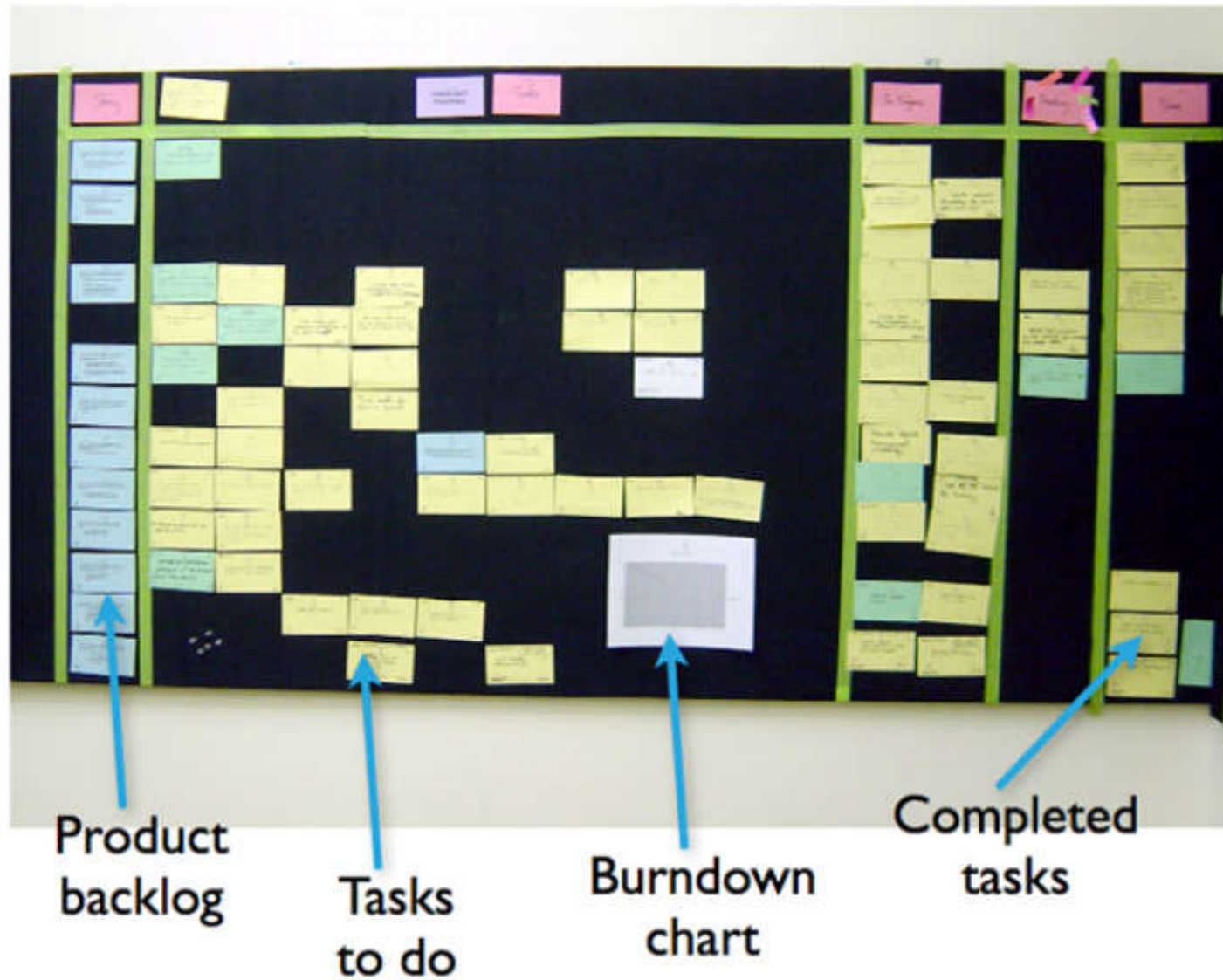
- Schlechte Implementierung,
- Unordentlicher Code ("mal aufräumen")
- Kleine Fehler
- Kaum getestet
- Internationalisierung fehlt

...

„Visual Workplace“



„Visual Workplace“



„Visual Workplace“



Zusammenfassung

- Scrum unterstützt eine agile Software-Entwicklung in einem dynamischen Umfeld, indem sich fortwährend Änderungen ergeben können.
- Scrum verzichtet nicht auf Planung, aber hält diese knapp und konkret auf die nahe Zukunft gerichtet
- Scrum setzt auf Selbstorganisation und Team-Dynamik

